

**Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Алтайский государственный технический университет
им. И.И. Ползунова» (АлтГТУ)**

Университетский технологический колледжава

Кафедра «Информационные системы в экономике»
Специальность 09.02.07 «Веб-дизайн разработчик»

Курсовой проект защищен с оценкой _____

(подпись руководителя работы) С.Ю. Фетисова
(инициалы, фамилия)

« ____ » _____ 20__ г.

КУРСОВОЙ ПРОЕКТ

Разработка приложения для автоматизации деятельности администратора гостиничного комплекса

(тема курсового проекта)

Пояснительная записка КП 09.02.07.07.000ПЗ

по дисциплине Основы алгоритмизации и программирования

Студент группы 1ВЕБ-42 Ларионов В.Г. _____ 07.05.2026
(группа) (фамилия, имя, отчество) (подпись) (дата)

Руководитель работы преподаватель _____ С.Ю. Фетисова
(ученое звание, должность) (подпись) (инициалы, фамилия)

БАРНАУЛ 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
1. Описание деятельности гостиничного комплекса.....	5
1.1. Описание функций и услуг.....	6
1.2. Организационная структура.....	6
1.3. Функциональная модель бизнес-процесса.....	7
1.4. Требования к функционалу.....	10
2. Проектирование и разработка.....	10
2.1. Проектная часть.....	10
2.1.1. Проектирование базы данных.....	10
.....	11
2.1.2. Проектирование интерфейса.....	12
2.1.3. Проектирование технического и программного обеспечения.....	14
2.2. Программная реализация.....	14
Заключение.....	19
Список литературы.....	20

Введение

В современных условиях развития информационных технологий автоматизация бизнес-процессов становится неотъемлемой частью эффективного управления предприятиями сферы услуг. Гостиничный бизнес не является исключением, так как требует постоянного контроля за бронированием номеров, учетом клиентов и предоставлением дополнительных услуг.

Целью данного курсового проекта является разработка приложения для автоматизации деятельности администратора гостиничного комплекса.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить деятельность гостиничного комплекса;
- описать бизнес-процессы обслуживания клиентов;
- выполнить функциональное моделирование системы;
- сформулировать требования к разрабатываемому приложению;
- спроектировать базу данных;
- разработать интерфейс пользователя;
- реализовать программную часть системы;
- провести тестирование и отладку;
- разработать руководство пользователя.

Автоматизация деятельности администратора позволяет снизить вероятность ошибок, повысить скорость обслуживания клиентов и улучшить качество предоставляемых услуг.

1. Описание деятельности гостиничного комплекса

Гостиничный комплекс «Уютный берег» представляет собой современное предприятие сферы услуг, предоставляющее клиентам услуги проживания и дополнительного сервиса.

Основной деятельностью гостиницы является размещение гостей в номерах различного уровня комфортности, а также предоставление дополнительных услуг, направленных на повышение качества обслуживания.

Гостиница предлагает номера следующих типов:

- стандартные;
- комфорт;
- люкс;
- президентские апартаменты.

Дополнительно клиентам предоставляются услуги:

- питание (завтрак, ужин);
- трансфер;
- SPA-процедуры;
- прачечная.

Основной задачей администратора является координация процессов заселения, бронирования, учета клиентов и контроля состояния номерного фонда.

1.1. Описание функций и услуг

Основными функциями гостиничного комплекса являются:

- предоставление услуг проживания;
- учет и регистрация гостей;
- управление бронированием номеров;
- расчет стоимости проживания;
- предоставление дополнительных услуг;
- ведение отчетности.

Клиентами гостиницы являются:

- туристы;
- деловые люди;
- семьи;
- командировочные сотрудники.

Каждая категория клиентов имеет свои потребности:

- удобство размещения;
- доступ к дополнительным услугам;
- оперативное обслуживание;
- комфорт и безопасность.

1.2. Организационная структура

Организационная структура гостиничного комплекса «Уютный берег», изображенная на рисунке 1, построена с учетом специфики предоставления услуг размещения и обслуживания клиентов. Она обеспечивает эффективное распределение обязанностей между сотрудниками и стабильную работу предприятия.

Директор осуществляет общее руководство гостиничным комплексом, определяет стратегию развития, принимает управленческие решения и контролирует финансовую деятельность. Он отвечает за организацию работы всех подразделений и качество предоставляемых услуг.

Администратор играет ключевую роль в работе гостиницы, так как непосредственно взаимодействует с клиентами. В его обязанности входит оформление бронирований, регистрация гостей, заселение и выселение, ведение базы данных, а также консультирование клиентов по услугам гостиницы. Кроме

того, администратор координирует работу персонала и решает возникающие вопросы.

Бухгалтерия занимается ведением финансового учета, контролем доходов и расходов, составлением отчетности и расчетом заработной платы сотрудников. Это обеспечивает прозрачность финансовой деятельности гостиницы.

Обслуживающий персонал отвечает за уборку номеров и поддержание чистоты на территории гостиницы. Их работа напрямую влияет на комфорт проживания гостей и общее впечатление от обслуживания.

Технический персонал обеспечивает исправную работу оборудования и инженерных систем, устраняет неисправности и проводит профилактические работы, что гарантирует бесперебойное функционирование гостиницы.

Таким образом, организационная структура гостиничного комплекса позволяет эффективно организовать работу персонала и обеспечить высокий уровень обслуживания клиентов.

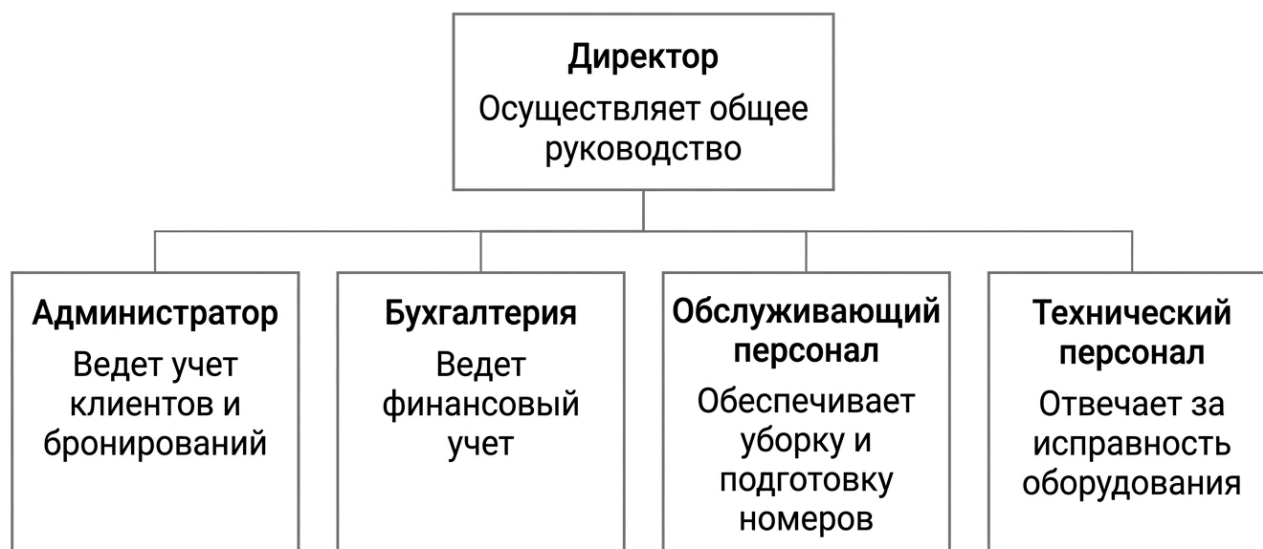


Рисунок 1 – Организационная структура гостиничного комплекса

1.3. Функциональная модель бизнес-процесса

Работа информационной системы в гостиничном комплексе направлена на автоматизацию основных процессов обслуживания клиентов, что значительно упрощает деятельность администратора. Система позволяет вносить и хранить данные о клиентах, управлять бронированиями, учитывать занятость номерного фонда и фиксировать предоставленные услуги. Это позволяет сократить время обработки информации и снизить вероятность ошибок, связанных с человеческим фактором, что в свою очередь повышает качество обслуживания.

На рисунке 2 изображена контекстная диаграмма приема клиентов. На данной диаграмме представлен бизнес-процесс взаимодействия администратора с клиентами гостиничного комплекса, начиная с момента обращения клиента и заканчивая оказанием услуг.

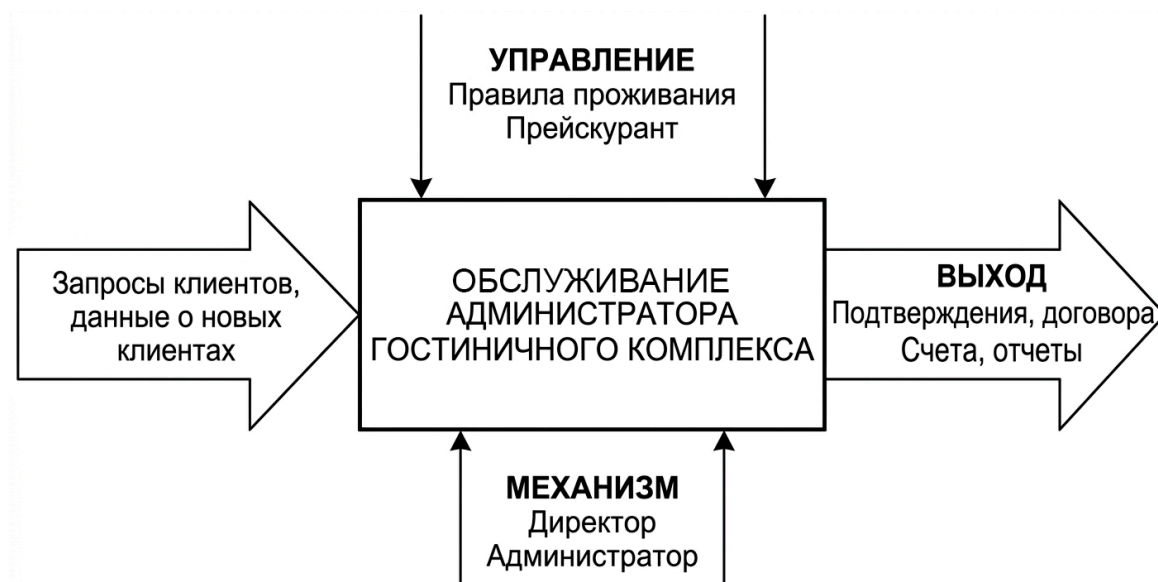


Рисунок 2 – Контекстная диаграмма приема клиентов

На рисунке 3 изображен полный процесс обслуживания клиентов в гостиничном комплексе. Диаграмма представляет собой поток информации в системе управления гостиницей.

Блок «Учет клиентов гостиничного комплекса»:

вход: клиент, документы клиента, запрос на проживание;

выход: зарегистрированный клиент, данные о клиенте, оформленное бронирование.

Блок «Оформление бронирования»:

вход: данные клиента, выбранный номер, даты проживания;

выход: подтвержденное бронирование, информация о заселении.

Блок «Предоставление услуг»:

вход: данные о бронировании, запрос на дополнительные услуги;

выход: оказанные услуги, информация о стоимости.

Блок «Расчет и оформление выезда»:

вход: данные о проживании и услугах;

выход: итоговый счет, заверщенное обслуживание клиента.

Потоки информации связаны между процессами и обеспечивают взаимодействие администратора с клиентами, базой данных и программной системой. Общие характеристики модели включают централизованное хранение данных, автоматизацию расчетов и поддержку принятия решений администратором.

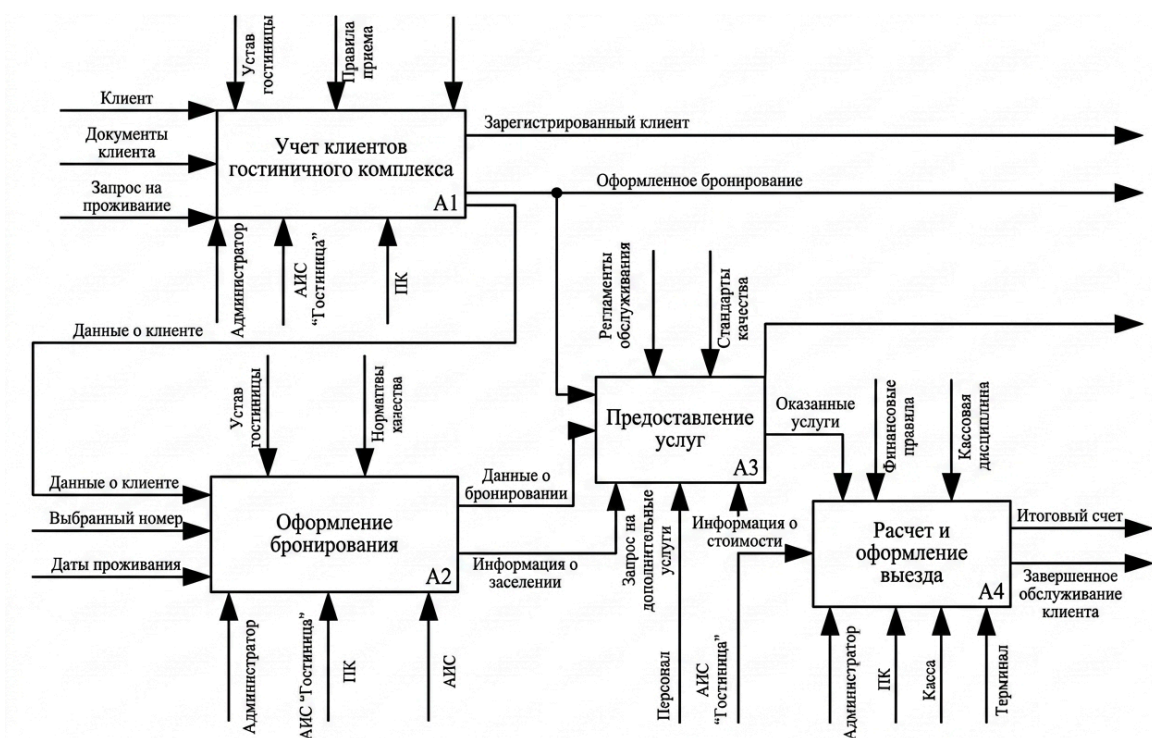


Рисунок 3 – Декомпозиция контекстной диаграммы в нотации IDEF0

1.4. Требования к функционалу

Система должна обеспечивать:

- добавление и редактирование данных о клиентах;
- управление номерным фондом;
- создание и обработку бронирований;
- учет дополнительных услуг;
- расчет стоимости проживания;
- формирование отчетов;
- удобный пользовательский интерфейс.

2. Проектирование и разработка

2.1. Проектная часть

2.1.1. Проектирование базы данных

В данной автоматизированной информационной системе используется база данных SQLite. База данных состоит из нескольких таблиц, предназначенных для хранения информации о клиентах, номерах, бронированиях и дополнительных услугах гостиничного комплекса.

В первой таблице хранятся данные о клиентах: идентификатор, фамилия, имя, отчество, телефон, электронная почта и паспортные данные. Атрибут «идентификатор» является первичным ключом и имеет числовой формат данных. Остальные поля имеют текстовый формат. Взаимодействие с данной таблицей осуществляется при добавлении, удалении и просмотре данных о клиентах.

Во второй таблице хранится информация о номерах гостиницы: идентификатор номера, тип, этаж, вместимость, стоимость и статус. Атрибут «идентификатор номера» является первичным ключом и имеет числовой формат. Данные используются при управлении номерным фондом и просмотре информации о номерах.

В третьей таблице хранится информация о бронированиях: идентификатор бронирования, идентификатор клиента, идентификатор номера, даты заезда и выезда, стоимость и статус. Таблица используется при оформлении и учете бронирований.

В четвертой таблице содержится информация о дополнительных услугах: идентификатор услуги, название и стоимость. Взаимодействие с таблицей осуществляется при добавлении и просмотре услуг.

В пятой таблице реализована связь между бронированиями и услугами, что позволяет учитывать оказанные услуги для каждого клиента и формировать итоговую стоимость.

Все взаимодействия с базой данных осуществляются через интерфейс автоматизированной информационной системы.

На рисунке 4 показана схема базы данных, используемой в приложении.

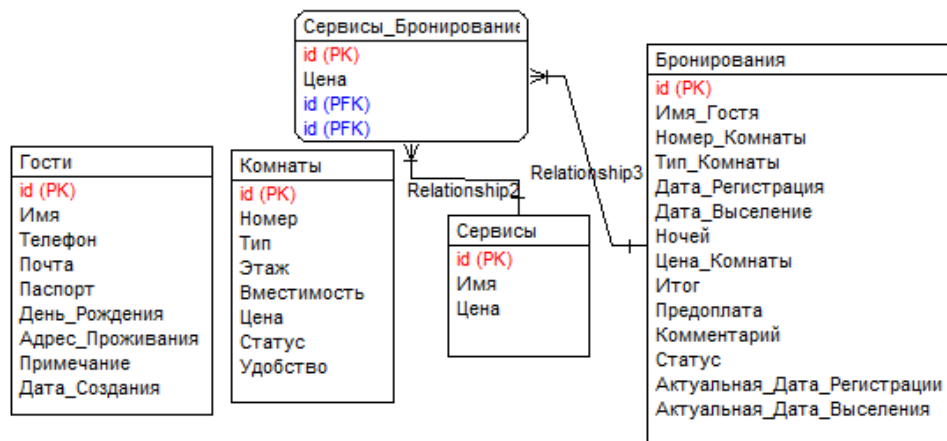


Рисунок 4 – Логическая структура базы данных

2.1.2. Проектирование интерфейса

Далее представлены макеты окон интерфейса на рисунках 5 - 9.

The mockup shows a web interface for a hotel management system. At the top is a blue header with the text "СИСТЕМА УПРАВЛЕНИЯ ОТЕЛЕМ- ПАНЕЛЬ УПРАВЛЕНИЯ" on the left and "DATE" on the right. Below the header is a navigation bar with five tabs: "Номера", "Бронирования", "Гости", "Услуги", and "Отчёты". The "Номера" tab is selected. Below the navigation bar is a section labeled "Фильтры" containing a table with two rows of filters. The first row has "Тип номера", "Статус", and a "Обновить" button. The second row has a "Добавить номер" button. Below the filters is a large table with six columns: "Номер", "Тип", "Этаж", "Вместимость", "Цена", and "Статус". The table is currently empty.

Рисунок 5 – Интерфейс вкладки «Номера»

The mockup shows a web interface for a hotel management system. At the top is a blue header with the text "СИСТЕМА УПРАВЛЕНИЯ ОТЕЛЕМ- ПАНЕЛЬ УПРАВЛЕНИЯ" on the left and "DATE" on the right. Below the header is a navigation bar with five tabs: "Номера", "Бронирования", "Гости", "Услуги", and "Отчёты". The "Бронирования" tab is selected. Below the navigation bar is a section containing three buttons: "Новое бронирование", "Обновить", and "Создать счёт". Below these buttons is a large table with eight columns: "ID", "Гость", "Номер", "Заезд", "Выезд", "Ночей", "Итого", and "Статус". The table is currently empty.

Рисунок 6 – Интерфейс вкладки «Бронирование»

СИСТЕМА УПРАВЛЕНИЯ ОТЕЛЕМ- ПАНЕЛЬ УПРАВЛЕНИЯ

DATE

НомераБронированияГостиУслугиОтчёты

Новый гость

-

-

Поиск

ID	ФИО	Телефон	Почта	Паспорт	Визитов	Последний визит
----	-----	---------	-------	---------	---------	-----------------

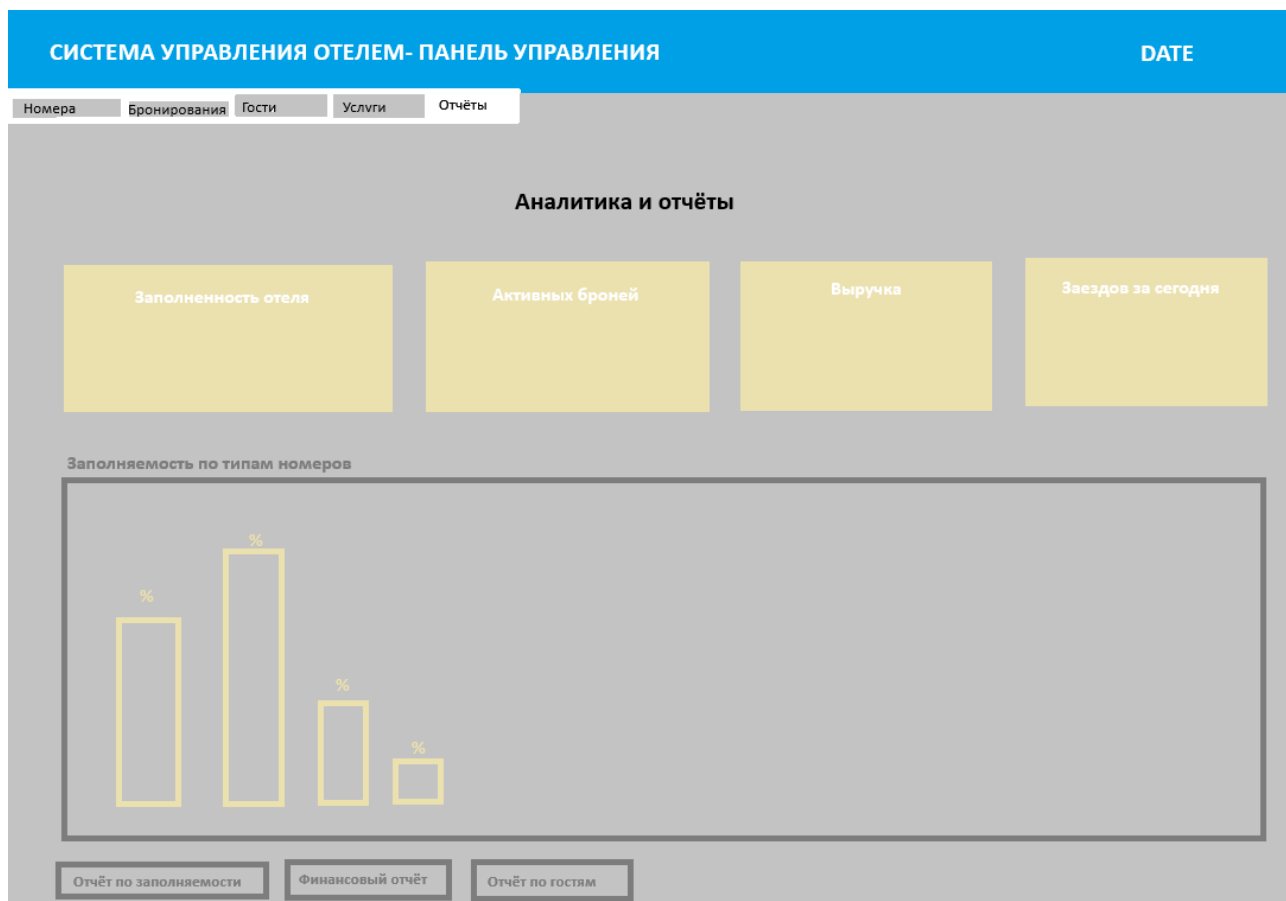


Рисунок 9 – Интерфейс вкладки «Отчеты»

2.1.3. Проектирование технического и программного обеспечения

Минимальные требования:

- процессор: Intel i3 и выше;
- оперативная память: от 4 ГБ;
- свободное место: от 1 ГБ;
- ОС: Windows 7 и выше;
- Python 3.9+;
- среда разработки: PyCharm / VS Code.

2.2. Программная реализация

Пример работы автоматизированной информационной системы представлен в виде руководства пользователя. Данное приложение предназначена для упрощения работы администратора гостиничного комплекса и повышения эффективности управления информацией о клиентах, бронированиях и предоставляемых услугах.

Система реализована на языке программирования Python с использованием библиотеки Tkinter, что позволяет создать удобный графический интерфейс

пользователя. Программа обеспечивает работу с данными, включая их добавление, редактирование и удаление, а также обработку пользовательских действий и формирование необходимой информации.

Для начала работы с приложением необходимо установить Python версии не ниже 3.9. После этого следует открыть среду разработки, например Visual Studio Code или PyCharm, выбрать папку с проектом и запустить основной файл программы.

В процессе работы пользователь взаимодействует с интерфейсом системы, который позволяет управлять данными о клиентах, номерах, бронированиях и услугах, что значительно упрощает выполнение основных задач администратора гостиничного комплекса.

2.2.1. Руководство пользователя

Для использования программы необходимо иметь документацию, описывающую функционал системы. Данная автоматизированная информационная система предназначена для управления деятельностью администратора гостиничного комплекса, включая учет клиентов, номеров, бронирований и дополнительных услуг.

Интерфейс программы реализован с использованием библиотеки Tkinter и состоит из нескольких вкладок, каждая из которых отвечает за определенный функционал системы.

Для работы с номерным фондом необходимо перейти во вкладку «Номера», показано на рисунке 10. В данной вкладке пользователь может добавлять новые номера, редактировать существующие данные, а также просматривать информацию о номерах, включая их тип, стоимость, вместимость и текущий статус. Для выполнения операций необходимо заполнить соответствующие поля и нажать кнопку действия.

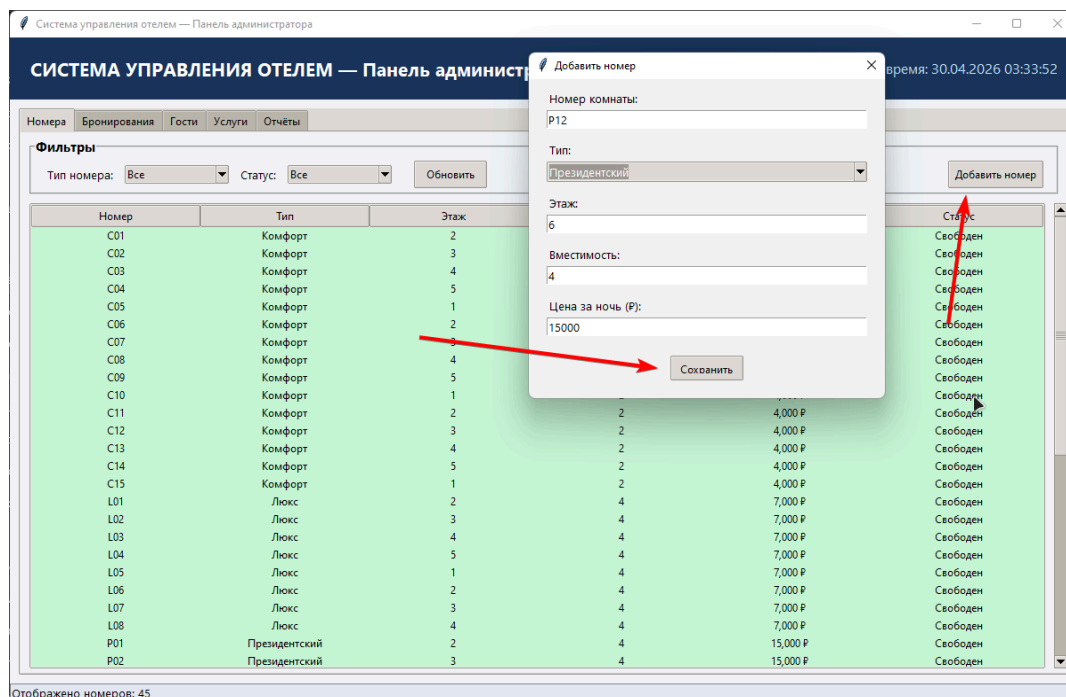


Рисунок 10 – Вкладка «Номера»

Работа с клиентами осуществляется во вкладке «Гости», показано на рисунке 11. В данной вкладке пользователь может добавить нового клиента, введя его персональные данные, а также редактировать или удалять существующие записи. Для выполнения действий необходимо выбрать соответствующую функцию и нажать кнопку.

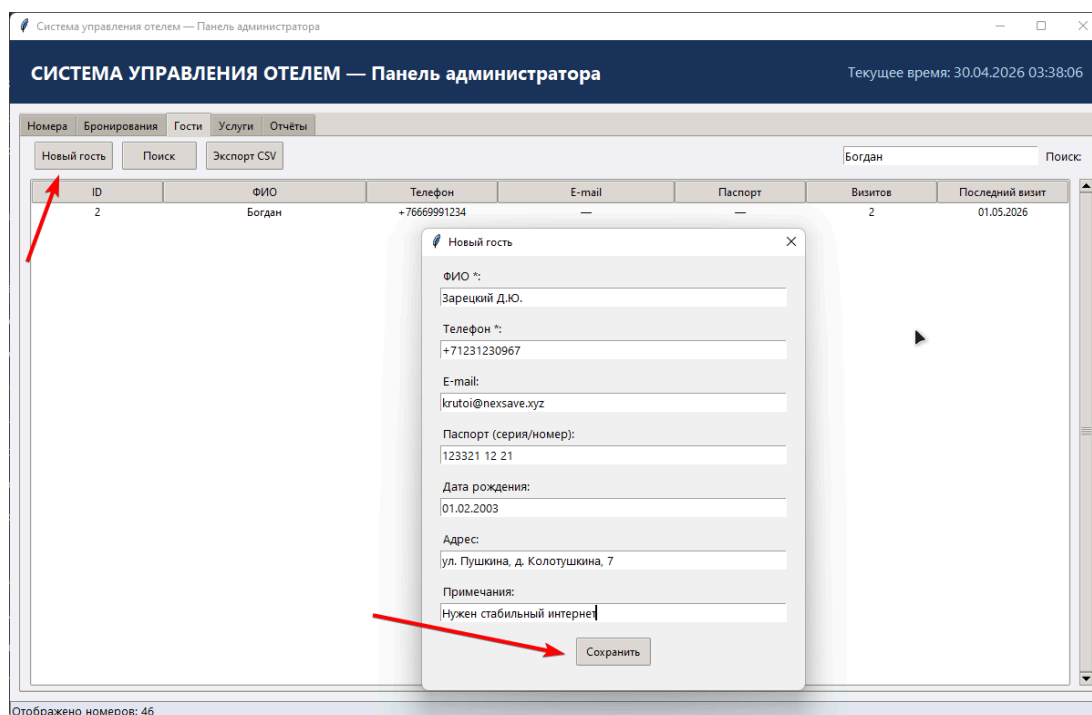


Рисунок 11 – Вкладка «Гости»

Оформление бронирований выполняется во вкладке «Бронирования», показано на рисунках 12-13. Пользователь выбирает клиента, номер, указывает даты заезда и выезда, после чего система автоматически рассчитывает стоимость проживания. Также в данной вкладке можно просматривать и изменять существующие бронирования.

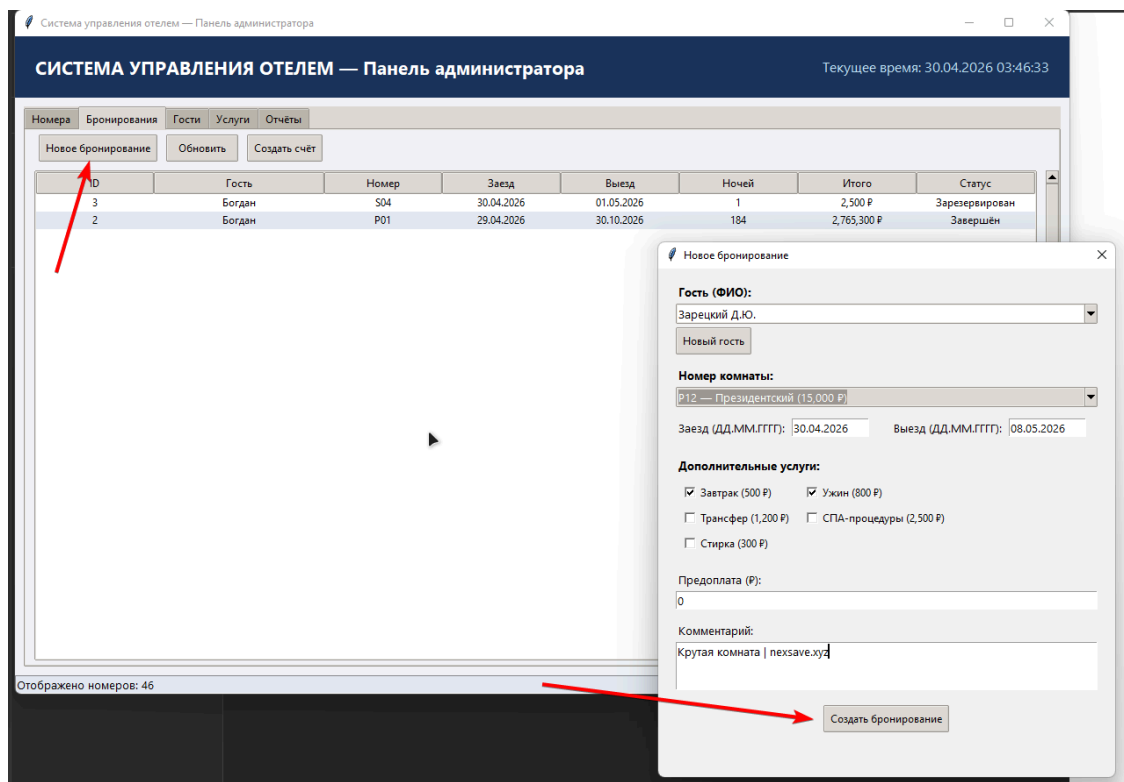


Рисунок 12 – Вкладка «Бронирования»

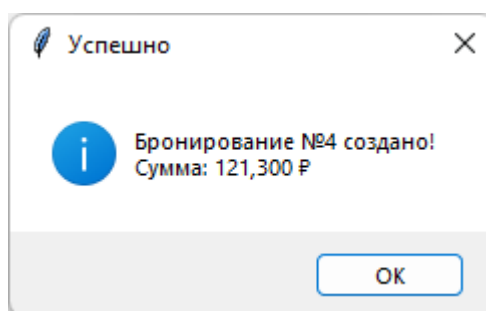


Рисунок 13 – Вкладка «Бронирования» успех

Управление дополнительными услугами осуществляется во вкладке «Услуги», показано на рисунке 14. В данной вкладке можно добавлять новые услуги, указывать их стоимость, а также редактировать или удалять существующие записи. Это позволяет учитывать все дополнительные сервисы, предоставляемые клиентам.

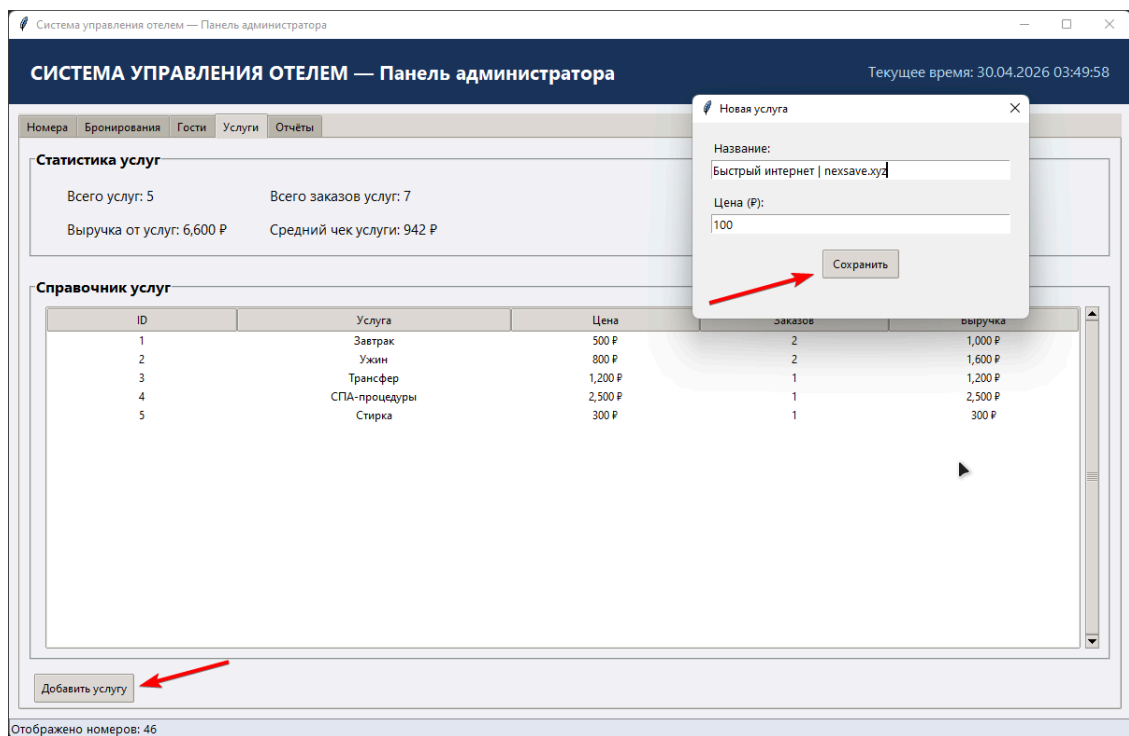


Рисунок 14 – Вкладка «Услуги»

Просмотр общей информации и формирование отчетов осуществляется во вкладке «Отчеты», показано на рисунке 15. Пользователь может получить данные о количестве клиентов, бронирований, а также общей загрузке гостиницы. Также пользователь может формировать отчёты, пример показан на рисунках 16-17.

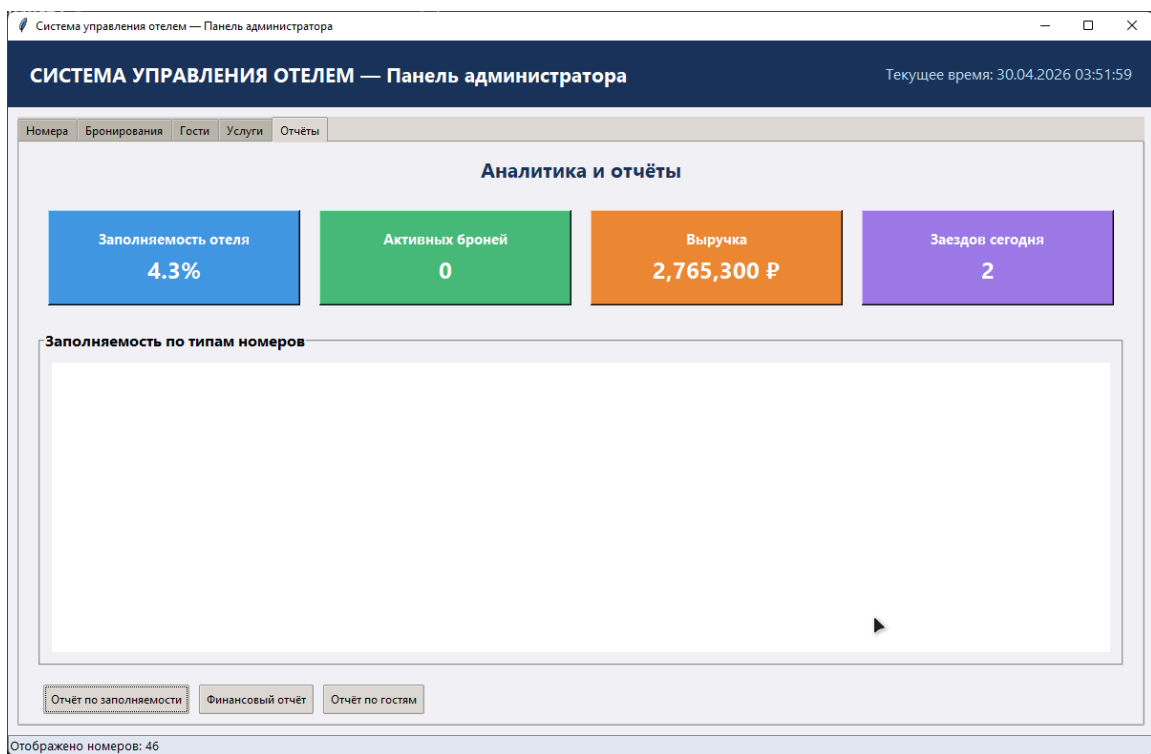


Рисунок 15 – Вкладка «Отчеты»

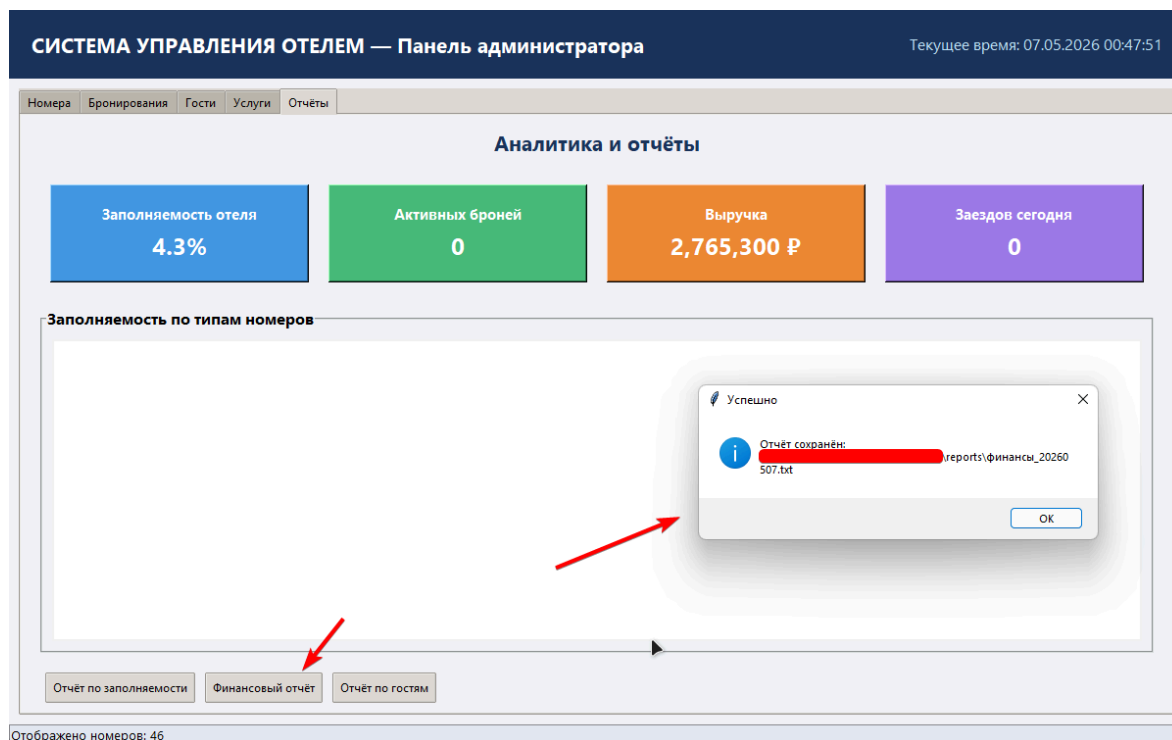


Рисунок 16 - Формирование отчета «Финансовый отчет»

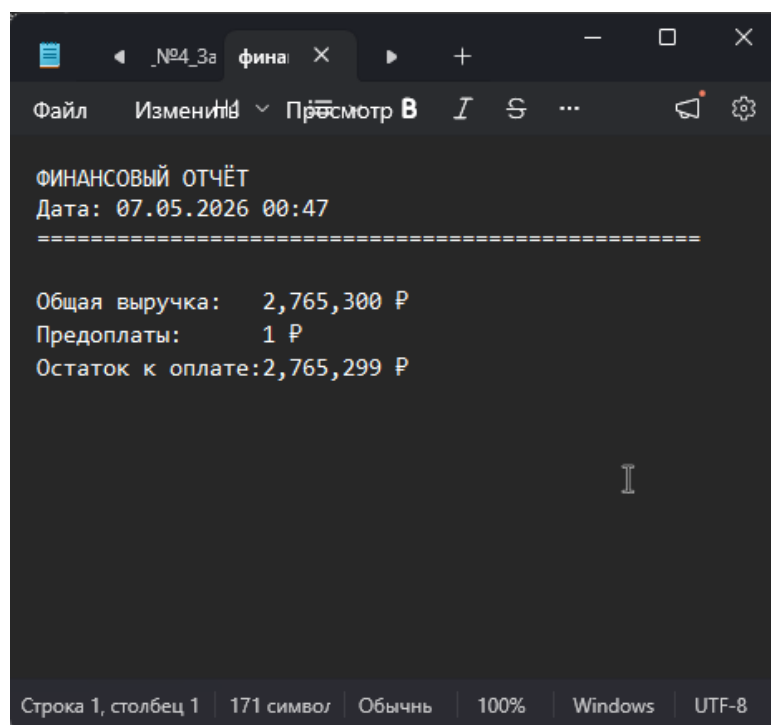


Рисунок 17 - Визуал отчета «Финансовый отчет»

Таким образом, разработанная система обеспечивает удобный интерфейс для работы администратора и позволяет эффективно управлять основными процессами гостиничного комплекса.

Заключение

В ходе выполнения курсового проекта были успешно решены поставленные задачи по разработке автоматизированной информационной системы для автоматизации деятельности администратора гостиничного комплекса.

В процессе работы были выполнены все этапы, предусмотренные во введении. В частности, была изучена деятельность гостиничного комплекса, что позволило определить основные бизнес-процессы, связанные с обслуживанием клиентов, управлением номерным фондом и предоставлением дополнительных услуг. Проведенное функциональное моделирование позволило наглядно представить процессы работы администратора и определить требования к разрабатываемой системе.

В результате была спроектирована и реализована автоматизированная информационная система, обеспечивающая учет клиентов, управление бронированиями, контроль занятости номеров и учет дополнительных услуг. Разработанная система позволяет сократить время обработки данных, снизить вероятность ошибок и повысить эффективность работы администратора.

Таким образом, создание приложения отвечает основным требованиям гостиничного комплекса и может быть использована для повышения качества обслуживания клиентов. Перспективы развития системы могут включать расширение функционала, внедрение онлайн-бронирования, интеграцию с платежными системами и развитие аналитических возможностей, что позволит повысить конкурентоспособность гостиницы и уровень предоставляемых услуг.

Список литературы

1. Лутц М. Изучаем Python. – 5-е изд. – СПб.: Диалектика, 2019. – 1360 с.
2. Романько В.К. Базы данных. – М.: Юрайт, 2020. – 380 с.
3. Официальная документация Python 3.9. – [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 10.04.2025).
4. Документация Tkinter. – [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 10.04.2025).
5. SQLite Documentation (зеркало). – [Электронный ресурс]. – Режим доступа: <https://sqliteonline.com/ru/docs/> (дата обращения: 10.04.2025).
6. Методические указания по выполнению курсового проекта. – Барнаул: АлтГТУ, 2025.
7. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.

Приложение А. Задание на курсовое проектирование

Министерство науки и высшего образования Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
(АлтГТУ)

Университетский технологический колледж

Кафедра

«Информационные системы в экономике»

ЗАДАНИЕ

на курсовой проект по дисциплине
«Основы алгоритмизации и программирования»

студенту группы 1ВЕБ-42 Ларионову Владиславу Геннадьевичу

Тема курсового проекта: «Разработка приложения для автоматизации деятельности администратора гостиничного комплекса»

Календарный план работы:

Но этапа	Содержание этапа	Неделя семестра
1	Получение задания	3
2	Анализ деятельности гостиничного комплекса. Оформление соответствующего раздела пояснительной записки	4
5	Проектирование основных элементов АИС (базы данных, интерфейса, технического и программного обеспечения). Оформление соответствующего раздела пояснительной записки	6

6	Разработка практической части. Проектирование бизнес-процесса приема клиентов фитнес-центр «как должно быть». Оформление соответствующего раздела пояснительной записки	8
7	Проектирование и разработка кода графического интерфейса пользователя. Оформление соответствующего раздела пояснительной записки	10
8	Разработка содержательной (основной) части программы. Оформление соответствующего раздела пояснительной записки	12
9	Разработка системы отчетов. Оформление соответствующего раздела пояснительной записки	14
10	Оформление пояснительной записки с приложениями и презентации	15
11	Защита курсового проекта	16

Руководитель работы _____ С.Ю. Фетисова

Дата выдачи задания « ____ » _____ 2026 г.

Задание принял к исполнению _____ В.Г. Ларионов

Приложение Б. Листинг кода

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
from datetime import datetime, timedelta
import sqlite3
import os
import csv

DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "hotel.db")

class DatabaseManager:

    def __init__(self, db_path: str):
        self.db_path = db_path
        self.conn = sqlite3.connect(db_path)
        self.conn.row_factory = sqlite3.Row
        self.conn.execute("PRAGMA foreign_keys = ON")
        self._create_tables()
        self._seed_services()
        self._seed_demo_rooms()

    def _create_tables(self):
        cur = self.conn.cursor()

        cur.executescript("""
            CREATE TABLE IF NOT EXISTS rooms (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                number        TEXT      NOT NULL UNIQUE,
                type           TEXT      NOT NULL,
                floor          INTEGER NOT NULL,
                capacity        INTEGER NOT NULL,
                price          INTEGER NOT NULL,
                status         TEXT      NOT NULL DEFAULT 'Свободен',
                amenities      TEXT      DEFAULT ''
            );

            CREATE TABLE IF NOT EXISTS guests (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                name           TEXT      NOT NULL,
                phone          TEXT      NOT NULL,
                email          TEXT      DEFAULT '',
                passport        TEXT      DEFAULT '',
                birthdate       TEXT      DEFAULT '',
                address         TEXT      DEFAULT ''
            );
        """)
```

```

        notes      TEXT      DEFAULT '',
        created    TEXT      DEFAULT ''
    );

    CREATE TABLE IF NOT EXISTS services (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        name        TEXT      NOT NULL UNIQUE,
        price       INTEGER NOT NULL
    );

    CREATE TABLE IF NOT EXISTS bookings (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        guest_name  TEXT      NOT NULL,
        room_number TEXT      NOT NULL,
        room_type   TEXT      NOT NULL DEFAULT '',
        check_in    TEXT      NOT NULL,
        check_out   TEXT      NOT NULL,
        nights      INTEGER NOT NULL DEFAULT 1,
        room_price  INTEGER NOT NULL DEFAULT 0,
        total       INTEGER NOT NULL DEFAULT 0,
        prepay      INTEGER NOT NULL DEFAULT 0,
        comment     TEXT      DEFAULT '',
        status      TEXT      NOT NULL DEFAULT 'Зарезервирован',
        created_at  TEXT      DEFAULT '',
        check_in_actual TEXT    DEFAULT '',
        check_out_actual TEXT    DEFAULT ''
    );

    CREATE TABLE IF NOT EXISTS booking_services (
        id          INTEGER PRIMARY KEY AUTOINCREMENT,
        booking_id  INTEGER NOT NULL REFERENCES bookings(id) ON DELETE
CASCADE,
        service_id INTEGER NOT NULL REFERENCES services(id),
        price       INTEGER NOT NULL
    );

    """)
    self.conn.commit()

def _seed_services(self):
    defaults = [
        ("Завтрак", 500),
        ("Ужин", 800),
        ("Трансфер", 1200),
        ("СПА-процедуры", 2500),
        ("Стирка", 300),

```



```

    ]
    cur = self.conn.cursor()
    for name, price in defaults:
        cur.execute("INSERT OR IGNORE INTO services (name, price) VALUES (?,
?)", (name, price))
    self.conn.commit()

def _seed_demo_rooms(self):
    cur = self.conn.cursor()
    if cur.execute("SELECT COUNT(*) FROM rooms").fetchone()[0] > 0:
        return

    room_types = [
        ("Стандарт", 2500, 20, 2),
        ("Комфорт", 4000, 15, 2),
        ("Люкс", 7000, 8, 4),
        ("Президентский", 15000, 2, 4),
    ]
    amenities_map = {
        "Стандарт": "Wi-Fi, ТВ",
        "Комфорт": "Wi-Fi, ТВ, Кондиционер",
        "Люкс": "Wi-Fi, ТВ, Кондиционер, Мини-бар",
        "Президентский": "Wi-Fi, ТВ, Кондиционер, Мини-бар, Джакузи",
    }
    prefix_map = {"Стандарт": "S", "Комфорт": "C", "Люкс": "L",
"Президентский": "P"}

    for rtype, price, count, cap in room_types:
        pfx = prefix_map[rtype]
        amen = amenities_map[rtype]
        for i in range(1, count + 1):
            floor = (i % 5) + 1
            number = f"{pfx}{i:02d}"
            cur.execute(
                "INSERT INTO rooms (number, type, floor, capacity, price,
status, amenities) "
                "VALUES (?, ?, ?, ?, ?, 'Свободен', ?)",
                (number, rtype, floor, cap, price, amen)
            )
        self.conn.commit()

def get_rooms(self, room_type=None, status=None):
    q = "SELECT * FROM rooms WHERE 1=1"
    params = []
    if room_type and room_type != "Все":

```

```

        q += " AND type = ?"
        params.append(room_type)
    if status and status != "Bce":
        q += " AND status = ?"
        params.append(status)
    q += " ORDER BY number"
    return self.conn.execute(q, params).fetchall()

    def add_room(self, number, rtype, floor, capacity, price,
amenities="Wi-Fi,TB"):
        self.conn.execute(
            "INSERT INTO rooms (number, type, floor, capacity, price, amenities)
VALUES (?, ?, ?, ?, ?, ?)",
            (number, rtype, floor, capacity, price, amenities)
        )
        self.conn.commit()

    def update_room_status(self, number, status):
        self.conn.execute("UPDATE rooms SET status = ? WHERE number = ?",
(status, number))
        self.conn.commit()

    def delete_room(self, number):
        self.conn.execute("DELETE FROM rooms WHERE number = ?", (number,))
        self.conn.commit()

    def get_free_rooms(self):
        return self.conn.execute(
            "SELECT * FROM rooms WHERE status = 'Свободен' ORDER BY number"
        ).fetchall()

    def get_guests(self, search=""):
        if search:
            like = f"%{search}%"
            return self.conn.execute(
                "SELECT * FROM guests WHERE name LIKE ? OR phone LIKE ? ORDER BY
name",
                (like, like)
            ).fetchall()
        return self.conn.execute("SELECT * FROM guests ORDER BY
name").fetchall()

    def add_guest(self, name, phone, email="", passport="", birthdate="",
address="", notes=""):
        created = datetime.now().strftime("%d.%m.%Y")

```

```

        self.conn.execute(
            "INSERT INTO guests (name, phone, email, passport, birthdate,
address, notes, created) "
            "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (name, phone, email, passport, birthdate, address, notes, created)
        )
        self.conn.commit()

    def get_guest_stats(self, guest_name):
        """Возвращает (количество_визитов, последний_выезд)."""
        rows = self.conn.execute(
            "SELECT check_out FROM bookings WHERE guest_name = ? ORDER BY id",
            (guest_name,)
        ).fetchall()
        visits = len(rows)
        last = rows[-1]["check_out"] if rows else "-"
        return visits, last

    def get_services(self):
        return self.conn.execute("SELECT * FROM services ORDER BY
id").fetchall()

    def add_service(self, name, price):
        self.conn.execute("INSERT INTO services (name, price) VALUES (?, ?)",
(name, price))
        self.conn.commit()

    def get_service_stats(self, service_id):
        """Возвращает (количество_заказов, выручка)."""
        row = self.conn.execute(
            "SELECT COUNT(*) as cnt, COALESCE(SUM(price), 0) as rev "
            "FROM booking_services WHERE service_id = ?",
            (service_id,)
        ).fetchone()
        return row["cnt"], row["rev"]

    def get_bookings(self):
        return self.conn.execute(
            "SELECT * FROM bookings ORDER BY id DESC"
        ).fetchall()

    def get_booking(self, booking_id):
        return self.conn.execute(
            "SELECT * FROM bookings WHERE id = ?", (booking_id,)
        ).fetchone()

```

```

def get_booking_services(self, booking_id):
    return self.conn.execute(
        "SELECT bs.id, bs.price, s.name "
        "FROM booking_services bs JOIN services s ON bs.service_id = s.id "
        "WHERE bs.booking_id = ?",
        (booking_id,)
    ).fetchall()

def add_booking(self, guest_name, room_number, room_type, check_in,
check_out,
                nights, room_price, total, prepay, comment, service_ids):
    created = datetime.now().strftime("%d.%m.%Y %H:%M")
    cur = self.conn.cursor()
    cur.execute(
        "INSERT INTO bookings (guest_name, room_number, room_type, check_in,
check_out, "
        "nights, room_price, total, prepay, comment, status, created_at) "
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'Зарезервирован', ?)",
        (guest_name, room_number, room_type, check_in, check_out,
        nights, room_price, total, prepay, comment, created)
    )
    booking_id = cur.lastrowid

    services = self.get_services()
    svc_map = {s["id"]: s for s in services}
    for sid in service_ids:
        if sid in svc_map:
            cur.execute(
                "INSERT INTO booking_services (booking_id, service_id,
price) VALUES (?, ?, ?)",
                (booking_id, sid, svc_map[sid]["price"])
            )

    self.conn.commit()
    return booking_id

def update_booking_status(self, booking_id, status, **extra):
    sets = ["status = ?"]
    vals = [status]
    for col, val in extra.items():
        sets.append(f"{col} = ?")
        vals.append(val)
    vals.append(booking_id)

```

```

        self.conn.execute(f"UPDATE bookings SET {'', ' '.join(sets)} WHERE id = ?",
vals)

        self.conn.commit()

    def check_room_conflict(self, room_number, check_in_dt, check_out_dt,
exclude_id=None):
        q = ("SELECT 1 FROM bookings WHERE room_number = ? AND status IN
('Активен', 'Зарезервирован') "
            "AND check_in < ? AND check_out > ?")
        params = [room_number, check_out_dt.strftime("%d.%m.%Y"),
check_in_dt.strftime("%d.%m.%Y")]
        if exclude_id:
            q += " AND id != ?"
            params.append(exclude_id)
        return bool(self.conn.execute(q, params).fetchone())

    def get_kpi(self):
        total_rooms = self.conn.execute("SELECT COUNT(*) FROM
rooms").fetchone()[0]
        occupied = self.conn.execute("SELECT COUNT(*) FROM rooms WHERE status
= 'Занят']").fetchone()[0]
        occ_rate = (occupied / total_rooms * 100) if total_rooms else 0

        total_rev = self.conn.execute(
            "SELECT COALESCE(SUM(total), 0) FROM bookings WHERE status IN
('Активен', 'Завершён') "
        ).fetchone()[0]

        active = self.conn.execute(
            "SELECT COUNT(*) FROM bookings WHERE status = 'Активен'"
        ).fetchone()[0]

        today = datetime.now().strftime("%d.%m.%Y")
        arrivals = self.conn.execute(
            "SELECT COUNT(*) FROM bookings WHERE check_in = ?", (today,)
        ).fetchone()[0]

        return occ_rate, active, total_rev, arrivals

    def get_room_type_occupancy(self):
        rows = self.conn.execute(
            "SELECT type, COUNT(*) as total, "
            "SUM(CASE WHEN status='Занят' THEN 1 ELSE 0 END) as occupied "
            "FROM rooms GROUP BY type"
        ).fetchall()

```

```

        return {r["type"]: {"total": r["total"], "occupied": r["occupied"]}} for
r in rows}

    def close(self):
        self.conn.close()

# ГЛАВНОЕ ПРИЛОЖЕНИЕ

class HotelAdminApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Система управления отелем — Панель администратора")
        self.root.geometry("1200x750")
        self.root.configure(bg="#f0f4f8")

        self.db = DatabaseManager(DB_PATH)

        self.setup_ui()
        self.update_all_views()

    # UI-каркас

    def setup_ui(self):
        style = ttk.Style()
        style.theme_use("clam")
        style.configure("Title.TLabel", font=("Segoe UI", 20, "bold"),
foreground="#1a365d")
        style.configure("Card.TFrame", background="white", relief="solid")
        style.configure("Accent.TButton", font=("Segoe UI", 11, "bold"),
background="#3182ce", foreground="white")

        header = tk.Frame(self.root, bg="#1a365d", height=70)
        header.pack(fill="x")
        header.pack_propagate(False)

        tk.Label(header, text="СИСТЕМА УПРАВЛЕНИЯ ОТЕЛЕМ — Панель
администратора",
font=("Segoe UI", 16, "bold"), bg="#1a365d",
fg="white").pack(side="left", padx=20, pady=10)

        self.clock_label = tk.Label(header, font=("Segoe UI", 12), bg="#1a365d",
fg="#bee3f8")
        self.clock_label.pack(side="right", padx=20, pady=10)
        self.update_clock()

```

```

self.notebook = ttk.Notebook(self.root)
self.notebook.pack(fill="both", expand=True, padx=10, pady=10)

self.tab_rooms = self._create_rooms_tab()
self.tab_bookings = self._create_bookings_tab()
self.tab_guests = self._create_guests_tab()
self.tab_services = self._create_services_tab()
self.tab_reports = self._create_reports_tab()

self.notebook.add(self.tab_rooms, text="Номера")
self.notebook.add(self.tab_bookings, text="Бронирования")
self.notebook.add(self.tab_guests, text="Гости")
self.notebook.add(self.tab_services, text="Услуги")
self.notebook.add(self.tab_reports, text="Отчёты")

self.status_bar = tk.Label(self.root, text="Готово", bd=1,
relief="sunken",
                                anchor="w", bg="#e2e8f0", font=("Segoe UI",
10))

self.status_bar.pack(side="bottom", fill="x")

def update_clock(self):
    now = datetime.now().strftime("%d.%m.%Y %H:%M:%S")
    self.clock_label.config(text=f"Текущее время: {now}")
    self.root.after(1000, self.update_clock)

# HOMEPA

def _create_rooms_tab(self):
    frame = tk.Frame(self.root, bg="#f0f4f8")

    filter_frame = tk.LabelFrame(frame, text="Фильтры", font=("Segoe UI",
11, "bold"),
                                bg="#f0f4f8", padx=10, pady=5)
    filter_frame.pack(fill="x", padx=10, pady=5)

    tk.Label(filter_frame, text="Тип номера:", bg="#f0f4f8",
            font=("Segoe UI", 10)).pack(side="left", padx=5)
    self.room_type_filter = ttk.Combobox(
        filter_frame,
        values=["Все", "Стандарт", "Комфорт", "Люкс", "Президентский"],
        width=16, state="readonly")
    self.room_type_filter.set("Все")
    self.room_type_filter.pack(side="left", padx=5)

```

```

        self.room_type_filter.bind("<<ComboboxSelected>>", lambda e:
self.update_rooms_view())

        tk.Label(filter_frame, text="Статус:", bg="#f0f4f8",
                  font=("Segoe UI", 10)).pack(side="left", padx=5)
        self.room_status_filter = ttk.Combobox(
            filter_frame,
            values=["Все", "Свободен", "Занят", "На уборке", "На обслуживании"],
            width=16, state="readonly")
        self.room_status_filter.set("Все")
        self.room_status_filter.pack(side="left", padx=5)
        self.room_status_filter.bind("<<ComboboxSelected>>", lambda e:
self.update_rooms_view())

        ttk.Button(filter_frame, text="Обновить",
                    command=self.update_rooms_view).pack(side="left", padx=20)
        ttk.Button(filter_frame, text="Добавить номер",
                    command=self._add_room_dialog).pack(side="right", padx=5)

        columns = ("number", "type", "floor", "capacity", "price", "status")
        self.rooms_tree = ttk.Treeview(frame, columns=columns, show="headings",
height=20)

        headings = {"number": "Номер", "type": "Тип", "floor": "Этаж",
                    "capacity": "Вместимость", "price": "Цена/ночь", "status":
"Статус"}
        for col, text in headings.items():
            self.rooms_tree.heading(col, text=text)
            self.rooms_tree.column(col, anchor="center", width=130)

        scrollbar = ttk.Scrollbar(frame, orient="vertical",
command=self.rooms_tree.yview)
        self.rooms_tree.configure(yscrollcommand=scrollbar.set)
        self.rooms_tree.pack(side="left", fill="both", expand=True, padx=10,
pady=5)
        scrollbar.pack(side="right", fill="y", pady=5)

        self.rooms_tree.bind("<Button-3>", self._show_room_context_menu)
        return frame

    def update_rooms_view(self):
        for item in self.rooms_tree.get_children():
            self.rooms_tree.delete(item)

        rooms = self.db.get_rooms(

```



```

        self.room_type_filter.get(),
        self.room_status_filter.get()
    )
    for room in rooms:
        tag = {"Свободен": "free", "Занят": "occupied",
              "На уборке": "cleaning"}.get(room["status"], "")
        self.rooms_tree.insert("", "end", values=(
            room["number"], room["type"], room["floor"],
            room["capacity"], f"{room['price']:,} ₽", room["status"]
        ), tags=(tag,))

    self.rooms_tree.tag_configure("free", background="#c6f6d5")
    self.rooms_tree.tag_configure("occupied", background="#fed7d7")
    self.rooms_tree.tag_configure("cleaning", background="#feebc8")
    self.status_bar.config(text=f"Отображено номеров:
{len(self.rooms_tree.get_children())}")

    def _show_room_context_menu(self, event):
        item = self.rooms_tree.identify_row(event.y)
        if item:
            self.rooms_tree.selection_set(item)
            menu = tk.Menu(self.root, tearoff=0)
            menu.add_command(label="Изменить статус",
command=self._change_room_status)
            menu.add_separator()
            menu.add_command(label="Удалить",
command=self._delete_room)
            menu.post(event.x_root, event.y_root)

    def _change_room_status(self):
        selected = self.rooms_tree.selection()
        if not selected:
            return
        values = self.rooms_tree.item(selected[0], "values")
        room_num = values[0]

        dialog = tk.Toplevel(self.root)
        dialog.title("Изменить статус")
        dialog.geometry("320x160")
        dialog.transient(self.root)
        dialog.grab_set()

        tk.Label(dialog, text=f"Home {room_num}", font=("Segoe UI", 12,
"bold")).pack(pady=10)
        status_var = tk.StringVar(value=values[5])
        ttk.Combobox(dialog, textvariable=status_var,

```

```

        values=["Свободен", "Занят", "На уборке", "На
обслуживании"],

        state="readonly").pack(pady=5)

def save():
    self.db.update_room_status(room_num, status_var.get())
    self.update_rooms_view()
    dialog.destroy()
    messagebox.showinfo("Успешно", "Статус обновлён!")

ttk.Button(dialog, text="Сохранить", command=save).pack(pady=10)

def _add_room_dialog(self):
    dialog = tk.Toplevel(self.root)
    dialog.title("Добавить номер")
    dialog.geometry("400x360")
    dialog.transient(self.root)
    dialog.grab_set()

    fields = [
        ("Номер комнаты:", "number", "entry", None),
        ("Тип:", "type", "combo",
         ["Стандарт", "Комфорт", "Люкс", "Президентский"]),
        ("Этаж:", "floor", "entry", None),
        ("Вместимость:", "capacity", "entry", None),
        ("Цена за ночь (₽):", "price", "entry", None),
    ]

    entries = {}
    for label, key, wtype, opts in fields:
        tk.Label(dialog, text=label, font=("Segoe UI", 10)).pack(
            anchor="w", padx=20, pady=(10, 0))
        if wtype == "entry":
            entries[key] = tk.Entry(dialog, font=("Segoe UI", 10))
        else:
            entries[key] = ttk.Combobox(dialog, values=opts,
                                         state="readonly", font=("Segoe UI",
10))

        entries[key].pack(fill="x", padx=20, pady=2)

def save():
    try:
        self.db.add_room(
            number=entries["number"].get().strip(),
            rtype=entries["type"].get(),
            floor=int(entries["floor"].get()),

```

```

        capacity=int(entries["capacity"].get()),
        price=int(entries["price"].get()),
    )
    self.update_rooms_view()
    dialog.destroy()
    messagebox.showinfo("Успешно", "Номер добавлен!")
except Exception as e:
    messagebox.showerror("Ошибка", f"Проверьте введённые
данные!\n{e}")

    ttk.Button(dialog, text="Сохранить", command=save).pack(pady=20)

def _delete_room(self):
    selected = self.rooms_tree.selection()
    if not selected:
        return
    values = self.rooms_tree.item(selected[0], "values")
    if messagebox.askyesno("Подтверждение", f"Удалить номер {values[0]}?"):
        self.db.delete_room(values[0])
        self.update_rooms_view()

# БРОНИРОВАНИЯ

def _create_bookings_tab(self):
    frame = tk.Frame(self.root, bg="#f0f4f8")

    ctrl = tk.Frame(frame, bg="#f0f4f8")
    ctrl.pack(fill="x", padx=10, pady=5)
    ttk.Button(ctrl, text="Новое бронирование",
               command=self._new_booking_dialog).pack(side="left", padx=5)
    ttk.Button(ctrl, text="Обновить",
               command=self.update_bookings_view).pack(side="left", padx=5)
    ttk.Button(ctrl, text="Создать счёт",
               command=self._create_invoice).pack(side="left", padx=5)

    columns = ("id", "guest", "room", "check_in", "check_out",
               "nights", "total", "status")
    self.bookings_tree = ttk.Treeview(frame, columns=columns,
                                      show="headings", height=20)
    heads = {"id": "ID", "guest": "Гость", "room": "Номер",
             "check_in": "Заезд", "check_out": "Выезд",
             "nights": "Ночей", "total": "Итого", "status": "Статус"}
    for col, text in heads.items():
        self.bookings_tree.heading(col, text=text)
        self.bookings_tree.column(col, anchor="center", width=100)

```

```

self.bookings_tree.column("guest", width=160)

sb = ttk.Scrollbar(frame, orient="vertical",
                    command=self.bookings_tree.yview)
self.bookings_tree.configure(yscrollcommand=sb.set)
self.bookings_tree.pack(side="left", fill="both", expand=True,
                        padx=10, pady=5)
sb.pack(side="right", fill="y", pady=5)

self.bookings_tree.bind("<Button-3>", self._show_booking_context_menu)
return frame

def update_bookings_view(self):
    for item in self.bookings_tree.get_children():
        self.bookings_tree.delete(item)

    for b in self.db.get_bookings():
        tag = {"Активен": "active", "Завершён": "completed",
              "Отменён": "cancelled"}.get(b["status"], "")
        self.bookings_tree.insert("", "end", values=(
            b["id"], b["guest_name"], b["room_number"],
            b["check_in"], b["check_out"], b["nights"],
            f"{b['total']:,} ₽", b["status"]
        ), tags=(tag,))

    self.bookings_tree.tag_configure("active", background="#c6f6d5")
    self.bookings_tree.tag_configure("completed", background="#e2e8f0")
    self.bookings_tree.tag_configure("cancelled", background="#fed7d7")

def _show_booking_context_menu(self, event):
    item = self.bookings_tree.identify_row(event.y)
    if item:
        self.bookings_tree.selection_set(item)
        menu = tk.Menu(self.root, tearoff=0)
        menu.add_command(label="Заселить гостя",
command=self._check_in_guest)
        menu.add_command(label="Выселить гостя",
command=self._check_out_guest)
        menu.add_command(label="Отменить",
command=self._cancel_booking)
        menu.add_separator()
        menu.add_command(label="Подробности",
command=self._show_booking_details)
        menu.post(event.x_root, event.y_root)

```

```

def _get_selected_booking_id(self):
    selected = self.bookings_tree.selection()
    if not selected:
        return None
    return int(self.bookings_tree.item(selected[0], "values")[0])

def _new_booking_dialog(self):
    dialog = tk.Toplevel(self.root)
    dialog.title("Новое бронирование")
    dialog.geometry("520x580")
    dialog.transient(self.root)
    dialog.grab_set()

    tk.Label(dialog, text="Гость (ФИО):",
              font=("Segoe UI", 10, "bold")).pack(anchor="w", padx=20,
pady=(15, 0))
    guests = self.db.get_guests()
    guest_combo = ttk.Combobox(dialog, values=[g["name"] for g in guests],
                                font=("Segoe UI", 10))
    guest_combo.pack(fill="x", padx=20, pady=2)

    ttk.Button(dialog, text="Новый гость",
                command=lambda: [dialog.destroy(),
                                self._new_guest_dialog()]).pack(anchor="w",
padx=20, pady=2)

    tk.Label(dialog, text="Номер комнаты:",
              font=("Segoe UI", 10, "bold")).pack(anchor="w", padx=20,
pady=(10, 0))
    free_rooms = self.db.get_free_rooms()
    room_values = [f"{r['number']} - {r['type']} ({r['price']:,} ₺)" for r in
free_rooms]
    room_combo = ttk.Combobox(dialog, values=room_values,
                                state="readonly", font=("Segoe UI", 10))
    room_combo.pack(fill="x", padx=20, pady=2)

    dates_frame = tk.Frame(dialog)
    dates_frame.pack(fill="x", padx=20, pady=10)
    tk.Label(dates_frame, text="Заезд (ДД.ММ.ГГГГ):",
              font=("Segoe UI", 10)).pack(side="left")
    check_in_entry = tk.Entry(dates_frame, width=12, font=("Segoe UI", 10))
    check_in_entry.insert(0, datetime.now().strftime("%d.%m.%Y"))
    check_in_entry.pack(side="left", padx=5)
    tk.Label(dates_frame, text="Выезд (ДД.ММ.ГГГГ):",
              font=("Segoe UI", 10)).pack(side="left", padx=(20, 0))

```

```

check_out_entry = tk.Entry(dates_frame, width=12, font=("Segoe UI", 10))
check_out_entry.insert(
    0, (datetime.now() + timedelta(days=1)).strftime("%d.%m.%Y"))
check_out_entry.pack(side="left", padx=5)

tk.Label(dialog, text="Дополнительные услуги:",
         font=("Segoe UI", 10, "bold")).pack(anchor="w", padx=20,
pady=(10, 0))

svc_frame = tk.Frame(dialog)
svc_frame.pack(fill="x", padx=20, pady=5)
service_vars = {}
all_services = self.db.get_services()
for i, svc in enumerate(all_services):
    var = tk.BooleanVar()
    tk.Checkbutton(svc_frame,
                   text=f"{svc['name']} ({svc['price']:,} ₺)",
                   variable=var,
                   font=("Segoe UI", 9)
                   ).grid(row=i // 2, column=i % 2, sticky="w",
                        padx=5, pady=2)
    service_vars[svc["id"]] = var

tk.Label(dialog, text="Предоплата (₺):",
         font=("Segoe UI", 10)).pack(anchor="w", padx=20, pady=(10, 0))
prepay_entry = tk.Entry(dialog, font=("Segoe UI", 10))
prepay_entry.insert(0, "0")
prepay_entry.pack(fill="x", padx=20, pady=2)

tk.Label(dialog, text="Комментарий:",
         font=("Segoe UI", 10)).pack(anchor="w", padx=20, pady=(10, 0))
comment_text = tk.Text(dialog, height=3, font=("Segoe UI", 10))
comment_text.pack(fill="x", padx=20, pady=2)

def save_booking():
    try:
        guest_name = guest_combo.get().strip()
        room_str = room_combo.get()
        if not guest_name or not room_str:
            messagebox.showerror("Ошибка", "Заполните все обязательные
поля!")

        return

        room_num = room_str.split(" - ")[0]
        room = next(r for r in free_rooms if r["number"] == room_num)

```

```

        ci = self._validate_date(check_in_entry.get())
        co = self._validate_date(check_out_entry.get())
        if not ci or not co:
            messagebox.showerror("Ошибка",
                                  "Неверный формат даты! Используйте
ДД.ММ.ГГГГ")

            return
        nights = (co - ci).days
        if nights <= 0:
            messagebox.showerror("Ошибка",
                                  "Дата выезда должна быть позже даты
заезда!")

            return

        if self.db.check_room_conflict(room_num, ci, co):
            messagebox.showerror("Ошибка",
                                  "Номер занят на выбранные даты!")

            return

        selected_svc_ids = [sid for sid, var in service_vars.items()
                             if var.get()]
        svc_map = {s["id"]: s for s in all_services}
        total = room["price"] * nights + sum(
            svc_map[sid]["price"] for sid in selected_svc_ids
        )
        prepay = int(prepay_entry.get() or 0)
        comment = comment_text.get("1.0", "end").strip()

        bid = self.db.add_booking(
            guest_name, room_num, room["type"],
            check_in_entry.get(), check_out_entry.get(),
            nights, room["price"], total, prepay,
            comment, selected_svc_ids
        )
        self.db.update_room_status(room_num, "Занят")
        self.update_all_views()
        dialog.destroy()
        messagebox.showinfo("Успешно",
                             f"Бронирование №{bid} создано!\nСумма:
{total:,} ₸")

        except Exception as e:
            messagebox.showerror("Ошибка", f"Ошибка создания
бронирования:\n{e}")

        ttk.Button(dialog, text="Создать бронирование",

```

```

        command=save_booking).pack(pady=15)

def _check_in_guest(self):
    bid = self._get_selected_booking_id()
    if bid is None:
        return
    now = datetime.now().strftime("%d.%m.%Y %H:%M")
    self.db.update_booking_status(bid, "Активен", check_in_actual=now)
    self.update_bookings_view()
    messagebox.showinfo("Успешно", "Гость заселён!")

def _check_out_guest(self):
    bid = self._get_selected_booking_id()
    if bid is None:
        return
    booking = self.db.get_booking(bid)
    now = datetime.now().strftime("%d.%m.%Y %H:%M")
    self.db.update_booking_status(bid, "Завершён", check_out_actual=now)
    self.db.update_room_status(booking["room_number"], "На уборке")
    self.update_all_views()
    messagebox.showinfo("Успешно",
                        "Гость выселен. Статус номера изменён на «На
уборке».")

def _cancel_booking(self):
    bid = self._get_selected_booking_id()
    if bid is None:
        return
    if messagebox.askyesno("Подтверждение", "Отменить бронирование?"):
        booking = self.db.get_booking(bid)
        self.db.update_booking_status(bid, "Отменён")
        self.db.update_room_status(booking["room_number"], "Свободен")
        self.update_all_views()

def _show_booking_details(self):
    bid = self._get_selected_booking_id()
    if bid is None:
        return
    b = self.db.get_booking(bid)
    svcs = self.db.get_booking_services(bid)

    dialog = tk.Toplevel(self.root)
    dialog.title(f"Бронирование №{bid}")
    dialog.geometry("420x450")
    dialog.transient(self.root)

```



```

lines = [
    f"БРОНИРОВАНИЕ №{b['id']}",
    "",
    f"Гость: {b['guest_name']}",
    f"Номер: {b['room_number']} ({b['room_type']})",
    f"Заезд: {b['check_in']}",
    f"Выезд: {b['check_out']}",
    f"Ночей: {b['nights']}",
    "",
    f"Стоимость номера: {b['room_price']:,} ₽/ночь",
    f"Итого за проживание: {b['room_price'] * b['nights']:,} ₽",
    "",
    "Услуги:",
]

if svcs:
    for s in svcs:
        lines.append(f"    • {s['name']}: {s['price']:,} ₽")
else:
    lines.append("    Нет дополнительных услуг")
lines += [
    "",
    f"Итого: {b['total']:,} ₽",
    f"Предоплата: {b['prepay']:,} ₽",
    f"Статус: {b['status']}",
]

tk.Label(dialog, text="\n".join(lines),
          font=("Consolas", 11), justify="left", anchor="w"
          ).pack(padx=20, pady=20, fill="both")

def _create_invoice(self):
    bid = self._get_selected_booking_id()
    if bid is None:
        messagebox.showwarning("Предупреждение",
                                "Выберите бронирование для создания счёта")
        return
    b = self.db.get_booking(bid)
    svcs = self.db.get_booking_services(bid)

    filename = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Текстовый файл", "*.txt"), ("Все файлы", "*.*")],
        initialfile=f"Счёт_№{bid}_{b['guest_name']}.txt"
    )

```

```

if not filename:
    return

with open(filename, "w", encoding="utf-8") as f:
    f.write(f"{'='*50}\n")
    f.write(f"                ГОСТИНИЧНЫЙ КОМПЛЕКС\n")
    f.write(f"{'='*50}\n\n")
    f.write(f"СЧЁТ № {bid:06d}\n")
    f.write(f"Дата: {datetime.now().strftime('%d.%m.%Y %H:%M')}\n\n")
    f.write(f"ГОСТЬ: {b['guest_name']}\n")
    f.write(f"НОМЕР: {b['room_number']} ({b['room_type']})\n")
    f.write(f"ПЕРИОД: {b['check_in']} – {b['check_out']}\n")
    f.write(f"НОЧЕЙ: {b['nights']}\n\n")
    f.write(f"{'-'*50}\n")
    f.write(f"ПРОЖИВАНИЕ:\n")
    f.write(f"    {b['nights']} ночей × {b['room_price']:,} ₸ = "
           f"{b['room_price'] * b['nights']:,} ₸\n\n")
    f.write("ДОПОЛНИТЕЛЬНЫЕ УСЛУГИ:\n")
    if svcs:
        for s in svcs:
            f.write(f"    {s['name']}: {s['price']:,} ₸\n")
    else:
        f.write("    Нет\n")
    f.write(f"\n{'-'*50}\n")
    f.write(f"ИТОГО К ОПЛАТЕ: {b['total']:,} ₸\n")
    f.write(f"ПРЕДОПЛАТА: {b['prepay']:,} ₸\n")
    f.write(f"К ДОПЛАТЕ: {b['total'] - b['prepay']:,} ₸\n\n")
    f.write(f"{'='*50}\n")
    f.write(f"        Спасибо, что выбрали наш отель!\n")
    f.write(f"{'='*50}\n")

messagebox.showinfo("Успешно", f"Счёт сохранён:\n{filename}")

# ГОСТИ

def _create_guests_tab(self):
    frame = tk.Frame(self.root, bg="#f0f4f8")

    ctrl = tk.Frame(frame, bg="#f0f4f8")
    ctrl.pack(fill="x", padx=10, pady=5)
    ttk.Button(ctrl, text="Новый гость",
               command=self._new_guest_dialog).pack(side="left", padx=5)
    ttk.Button(ctrl, text="Поиск",
               command=self.update_guests_view).pack(side="left", padx=5)
    ttk.Button(ctrl, text="Экспорт CSV",

```

```

        command=self._export_guests).pack(side="left", padx=5)

self.guest_search_var = tk.StringVar()
tk.Label(ctrl, text="Поиск:", bg="#f0f4f8",
        font=("Segoe UI", 10)).pack(side="right")
tk.Entry(ctrl, textvariable=self.guest_search_var,
        font=("Segoe UI", 10), width=30).pack(side="right", padx=5)

columns = ("id", "name", "phone", "email", "passport", "visits",
"last_visit")
self.guests_tree = ttk.Treeview(frame, columns=columns,
                                show="headings", height=20)
heads = {"id": "ID", "name": "ФИО", "phone": "Телефон", "email":
"E-mail",
        "passport": "Паспорт", "visits": "Визитов",
        "last_visit": "Последний визит"}
for col, text in heads.items():
    self.guests_tree.heading(col, text=text)
    self.guests_tree.column(col, anchor="center", width=100)
self.guests_tree.column("name", width=180)
self.guests_tree.column("email", width=150)

sb = ttk.Scrollbar(frame, orient="vertical",
                    command=self.guests_tree.yview)
self.guests_tree.configure(yscrollcommand=sb.set)
self.guests_tree.pack(side="left", fill="both", expand=True,
                       padx=10, pady=5)
sb.pack(side="right", fill="y", pady=5)
return frame

def update_guests_view(self):
    for item in self.guests_tree.get_children():
        self.guests_tree.delete(item)

    search = self.guest_search_var.get().strip()
    for g in self.db.get_guests(search):
        visits, last = self.db.get_guest_stats(g["name"])
        self.guests_tree.insert("", "end", values=(
            g["id"], g["name"], g["phone"],
            g["email"] or "-", g["passport"] or "-",
            visits, last
        ))

def _new_guest_dialog(self):
    dialog = tk.Toplevel(self.root)

```

```

dialog.title("Новый гость")
dialog.geometry("420x420")
dialog.transient(self.root)
dialog.grab_set()

fields = [
    ("ФИО *:", "name"),
    ("Телефон *:", "phone"),
    ("E-mail:", "email"),
    ("Паспорт (серия/номер):", "passport"),
    ("Дата рождения:", "birthdate"),
    ("Адрес:", "address"),
    ("Примечания:", "notes"),
]

entries = {}
for label, key in fields:
    tk.Label(dialog, text=label,
              font=("Segoe UI", 10)).pack(anchor="w", padx=20, pady=(10,
0))

    entries[key] = tk.Entry(dialog, font=("Segoe UI", 10))
    entries[key].pack(fill="x", padx=20, pady=2)

def save():
    if not entries["name"].get().strip() or not
entries["phone"].get().strip():
        messagebox.showerror("Ошибка", "ФИО и Телефон обязательны!")
        return
    self.db.add_guest(
        name=entries["name"].get().strip(),
        phone=entries["phone"].get().strip(),
        email=entries["email"].get().strip(),
        passport=entries["passport"].get().strip(),
        birthdate=entries["birthdate"].get().strip(),
        address=entries["address"].get().strip(),
        notes=entries["notes"].get().strip(),
    )
    self.update_guests_view()
    dialog.destroy()
    messagebox.showinfo("Успешно", "Гость добавлен!")

ttk.Button(dialog, text="Сохранить", command=save).pack(pady=15)

def _export_guests(self):
    filename = filedialog.asksaveasfilename(
        defaultextension=".csv",

```

```

        filetypes=[("CSV файл", "*.csv")],
        initialfile="гости_экспорт.csv"
    )
    if not filename:
        return
    with open(filename, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f, delimiter=";")
        w.writerow(["ID", "ФИО", "Телефон", "E-mail",
                    "Паспорт", "Визитов", "Последний визит"])
        for g in self.db.get_guests():
            visits, last = self.db.get_guest_stats(g["name"])
            w.writerow([g["id"], g["name"], g["phone"],
                        g["email"], g["passport"], visits, last])
    messagebox.showinfo("Успешно", f"Экспорт завершён:\n{filename}")

# УСЛУГИ

def _create_services_tab(self):
    frame = tk.Frame(self.root, bg="#f0f4f8")

    stats_frame = tk.LabelFrame(frame, text="Статистика услуг",
                                 font=("Segoe UI", 12, "bold"),
                                 bg="#f0f4f8", padx=15, pady=10)
    stats_frame.pack(fill="x", padx=10, pady=10)
    self.services_stats_labels = []
    for i in range(4):
        lbl = tk.Label(stats_frame, text="-",
                       font=("Segoe UI", 11), bg="#f0f4f8")
        lbl.grid(row=i // 2, column=i % 2, sticky="w", padx=20, pady=5)
        self.services_stats_labels.append(lbl)

    list_frame = tk.LabelFrame(frame, text="Справочник услуг",
                                font=("Segoe UI", 12, "bold"),
                                bg="#f0f4f8", padx=10, pady=5)
    list_frame.pack(fill="both", expand=True, padx=10, pady=10)

    columns = ("id", "name", "price", "usage_count", "revenue")
    self.services_tree = ttk.Treeview(list_frame, columns=columns,
                                       show="headings", height=15)
    heads = {"id": "ID", "name": "Услуга", "price": "Цена",
             "usage_count": "Заказов", "revenue": "Выручка"}
    for col, text in heads.items():
        self.services_tree.heading(col, text=text)
        self.services_tree.column(col, anchor="center", width=130)
    self.services_tree.column("name", width=220)

```

```

sb = ttk.Scrollbar(list_frame, orient="vertical",
                    command=self.services_tree.yview)
self.services_tree.configure(yscrollcommand=sb.set)
self.services_tree.pack(side="left", fill="both", expand=True,
                        padx=5, pady=5)
sb.pack(side="right", fill="y", pady=5)

ctrl = tk.Frame(frame, bg="#f0f4f8")
ctrl.pack(fill="x", padx=10, pady=5)
ttk.Button(ctrl, text="Добавить услугу",
            command=self._add_service_dialog).pack(side="left", padx=5)
return frame

def update_services_view(self):
    for item in self.services_tree.get_children():
        self.services_tree.delete(item)

    total_rev = 0
    total_usage = 0

    for svc in self.db.get_services():
        cnt, rev = self.db.get_service_stats(svc["id"])
        total_rev += rev
        total_usage += cnt
        self.services_tree.insert("", "end", values=(
            svc["id"], svc["name"],
            f"{svc['price']:,} ₽", cnt, f"{rev:,} ₽"
        ))

    texts = [
        f"Всего услуг: {len(self.services_tree.get_children())}",
        f"Всего заказов услуг: {total_usage}",
        f"Выручка от услуг: {total_rev:,} ₽",
        f"Средний чек услуги: {total_rev // max(total_usage, 1):,} ₽",
    ]
    for lbl, t in zip(self.services_stats_labels, texts):
        lbl.config(text=t)

def _add_service_dialog(self):
    dialog = tk.Toplevel(self.root)
    dialog.title("Новая услуга")
    dialog.geometry("360x210")
    dialog.transient(self.root)
    dialog.grab_set()

```

```

tk.Label(dialog, text="Название:",
          font=("Segoe UI", 10)).pack(anchor="w", padx=20, pady=(15, 0))
name_entry = tk.Entry(dialog, font=("Segoe UI", 10))
name_entry.pack(fill="x", padx=20, pady=2)

tk.Label(dialog, text="Цена (₽):",
          font=("Segoe UI", 10)).pack(anchor="w", padx=20, pady=(10, 0))
price_entry = tk.Entry(dialog, font=("Segoe UI", 10))
price_entry.pack(fill="x", padx=20, pady=2)

def save():
    try:
        self.db.add_service(name_entry.get().strip(),
                             int(price_entry.get()))
        self.update_services_view()
        dialog.destroy()
        messagebox.showinfo("Успешно", "Услуга добавлена!")
    except ValueError:
        messagebox.showerror("Ошибка", "Неверная цена!")

ttk.Button(dialog, text="Сохранить", command=save).pack(pady=15)

# ОТЧЁТЫ

def _create_reports_tab(self):
    frame = tk.Frame(self.root, bg="#f0f4f8")

    tk.Label(frame, text="Аналитика и отчёты",
              font=("Segoe UI", 16, "bold"), bg="#f0f4f8",
              fg="#1a365d").pack(pady=10)

    kpi_frame = tk.Frame(frame, bg="#f0f4f8")
    kpi_frame.pack(fill="x", padx=20, pady=10)

    self.kpi_cards = []
    kpi_colors = ["#4299e1", "#48bb78", "#ed8936", "#9f7aea"]
    for i in range(4):
        card = tk.Frame(kpi_frame, bg=kpi_colors[i],
                        width=250, height=100, relief="raised", bd=2)
        card.grid(row=0, column=i, padx=10, pady=5, sticky="nsew")
        card.pack_propagate(False)
        t = tk.Label(card, text="—", font=("Segoe UI", 11, "bold"),
                      bg=kpi_colors[i], fg="white")
        t.pack(pady=(15, 0))

```

```

        v = tk.Label(card, text="-", font=("Segoe UI", 18, "bold"),
                      bg=kpi_colors[i], fg="white")
        v.pack()
        self.kpi_cards.append((t, v))
    for i in range(4):
        kpi_frame.grid_columnconfigure(i, weight=1)

    chart_frame = tk.LabelFrame(frame, text="Заполняемость по типам
номеров",
                                font=("Segoe UI", 12, "bold"), bg="#f0f4f8")
    chart_frame.pack(fill="both", expand=True, padx=20, pady=10)
    self.occupancy_canvas = tk.Canvas(chart_frame, bg="white", height=250)
    self.occupancy_canvas.pack(fill="both", expand=True, padx=10, pady=10)

    btn_frame = tk.Frame(frame, bg="#f0f4f8")
    btn_frame.pack(fill="x", padx=20, pady=10)
    ttk.Button(btn_frame, text="Отчёт по заполняемости",
               command=self._report_occupancy).pack(side="left", padx=5)
    ttk.Button(btn_frame, text="Финансовый отчёт",
               command=self._report_financial).pack(side="left", padx=5)
    ttk.Button(btn_frame, text="Отчёт по гостям",
               command=self._report_guests).pack(side="left", padx=5)

    return frame

def update_reports_view(self):
    occ, active, rev, arrivals = self.db.get_kpi()
    kpi_data = [
        ("Заполняемость отеля", f"{occ:.1f}%"),
        ("Активных броней",      str(active)),
        ("Выручка",              f"{rev:,} ₽"),
        ("Заездов сегодня",      str(arrivals)),
    ]
    for (t, v), (title, value) in zip(self.kpi_cards, kpi_data):
        t.config(text=title)
        v.config(text=value)

    self.occupancy_canvas.delete("all")
    room_types = self.db.get_room_type_occupancy()
    if not room_types:
        return

    cw = self.occupancy_canvas.winfo_width() or 800
    ch = self.occupancy_canvas.winfo_height() or 250
    bar_w = min(100, (cw - 100) // max(len(room_types), 1))

```



```

spacing = 30
start_x = 50
max_h = ch - 80
colors = {"Стандарт": "#4299e1", "Комфорт": "#48bb78",
          "Люкс": "#ed8936", "Президентский": "#9f7aea"}

for i, (rtype, data) in enumerate(room_types.items()):
    rate = (data["occupied"] / data["total"]) if data["total"] else 0
    bar_h = rate * max_h
    x = start_x + i * (bar_w + spacing)
    y = ch - 50 - bar_h
    self.occupancy_canvas.create_rectangle(
        x, y, x + bar_w, ch - 50,
        fill=colors.get(rtype, "#718096"), outline="white", width=2)
    self.occupancy_canvas.create_text(
        x + bar_w / 2, y - 15, text=f"{rate*100:.0f}%",
        font=("Segoe UI", 11, "bold"), fill="#2d3748")
    self.occupancy_canvas.create_text(
        x + bar_w / 2, ch - 35, text=rtype,
        font=("Segoe UI", 10), fill="#4a5568")
    self.occupancy_canvas.create_text(
        x + bar_w / 2, ch - 20,
        text=f"{data['occupied']}/{data['total']}",
        font=("Segoe UI", 9), fill="#718096")

def _report_occupancy(self):
    reports_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"reports")
    os.makedirs(reports_dir, exist_ok=True)
    filename = os.path.join(reports_dir,
f"заполняемость_{datetime.now().strftime('%Y%m%d')}.txt")
    with open(filename, "w", encoding="utf-8") as f:
        f.write(f"ОТЧЁТ ПО ЗАПОЛНЯЕМОСТИ\n")
        f.write(f"Дата: {datetime.now().strftime('%d.%m.%Y %H:%M')}\n")
        f.write("=" * 50 + "\n\n")
        for rtype, data in self.db.get_room_type_occupancy().items():
            rate = data["occupied"] / data["total"] * 100 if data["total"]
else 0
            f.write(f"{rtype}: {data['occupied']}/{data['total']}
({rate:.1f}%) \n")
        messagebox.showinfo("Успешно", f"Отчёт сохранён:\n{filename}")

def _report_financial(self):

```

```

        reports_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"reports")

        os.makedirs(reports_dir, exist_ok=True)
        filename = os.path.join(reports_dir,

f"финансы_{datetime.now().strftime('%Y%m%d')}.txt")
        conn = self.db.conn
        total = conn.execute(
            "SELECT COALESCE(SUM(total),0) FROM bookings "
            "WHERE status IN ('Активен','Завершён')"
        ).fetchone()[0]
        prepay = conn.execute(
            "SELECT COALESCE(SUM(prepay),0) FROM bookings"
        ).fetchone()[0]
        with open(filename, "w", encoding="utf-8") as f:
            f.write(f"ФИНАНСОВЫЙ ОТЧЁТ\n")
            f.write(f"Дата: {datetime.now().strftime('%d.%m.%Y %H:%M')}\n")
            f.write("=" * 50 + "\n\n")
            f.write(f"Общая выручка: {total:,} ₸\n")
            f.write(f"Предоплаты: {prepay:,} ₸\n")
            f.write(f"Остаток к оплате:{total - prepay:,} ₸\n")
        messagebox.showinfo("Успешно", f"Отчёт сохранён:\n{filename}")

    def _report_guests(self):
        reports_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"reports")

        os.makedirs(reports_dir, exist_ok=True)
        filename = os.path.join(reports_dir,

f"гости_{datetime.now().strftime('%Y%m%d')}.csv")
        with open(filename, "w", newline="", encoding="utf-8-sig") as f:
            w = csv.writer(f, delimiter=";")
            w.writerow(["ФИО", "Телефон", "Визитов", "Потрачено", "Последний
визит"])

            for g in self.db.get_guests():
                visits, last = self.db.get_guest_stats(g["name"])
                spent = self.db.conn.execute(
                    "SELECT COALESCE(SUM(total),0) FROM bookings WHERE
guest_name=?",
                    (g["name"],)
                ).fetchone()[0]
                w.writerow([g["name"], g["phone"], visits, spent, last])
            messagebox.showinfo("Успешно", f"Отчёт сохранён:\n{filename}")

    @staticmethod

```

```

def _validate_date(date_str):
    try:
        return datetime.strptime(date_str.strip(), "%d.%m.%Y")
    except ValueError:
        return None

def update_all_views(self):
    self.update_rooms_view()
    self.update_bookings_view()
    self.update_guests_view()
    self.update_services_view()
    self.update_reports_view()

def on_close(self):
    self.db.close()
    self.root.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = HotelAdminApp(root)
    root.protocol("WM_DELETE_WINDOW", app.on_close)
    root.mainloop()

```

Приложение В. Презентация

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж им. В.В. Петрова

КУРСОВОЙ ПРОЕКТ
КР 09.02.07.07.000ПЗ

Разработка приложения для автоматизации деятельности администратора гостиничного комплекса

Выполнил: студент гр. 1Веб-42 - Ларионов В.Г.

Руководитель: преподаватель каф. ИСЭ - Фетисова С.Ю.

БАРНАУЛ 2025

Рисунок 18 – Титульный слайд

Цели и задачи проекта

Целью данного курсового проекта является разработка приложения для автоматизации деятельности администратора гостиничного комплекса.

Задачи:

- изучить деятельность гостиничного комплекса;
- описать бизнес-процессы обслуживания клиентов;
- выполнить функциональное моделирование системы;
- сформулировать требования к разрабатываемому приложению;
- спроектировать базу данных;
- разработать интерфейс пользователя;
- реализовать программную часть системы;
- провести тестирование и отладку;
- разработать руководство пользователя.

2

Рисунок 19 – Цели и Задачи

Описание деятельности гостиничного комплекса

Гостиничный комплекс «Уютный берег» представляет собой современное предприятие сферы услуг, предоставляющее клиентам услуги проживания и дополнительного сервиса.

Основной деятельностью гостиницы является размещение гостей в номерах различного уровня комфортности, а также предоставление дополнительных услуг, направленных на повышение качества обслуживания.

Основной задачей администратора является координация процессов заселения, бронирования, учета клиентов и контроля состояния номерного фонда.

3

Рисунок 20 – Описание деятельности

Организационная структура



4

Рисунок 21 – Организационная структура

Об организационной структуре

Организационная структура гостиничного комплекса «Уютный берег» построена с учетом специфики предоставления услуг размещения и обслуживания клиентов. Она обеспечивает эффективное распределение обязанностей между сотрудниками и стабильную работу предприятия.

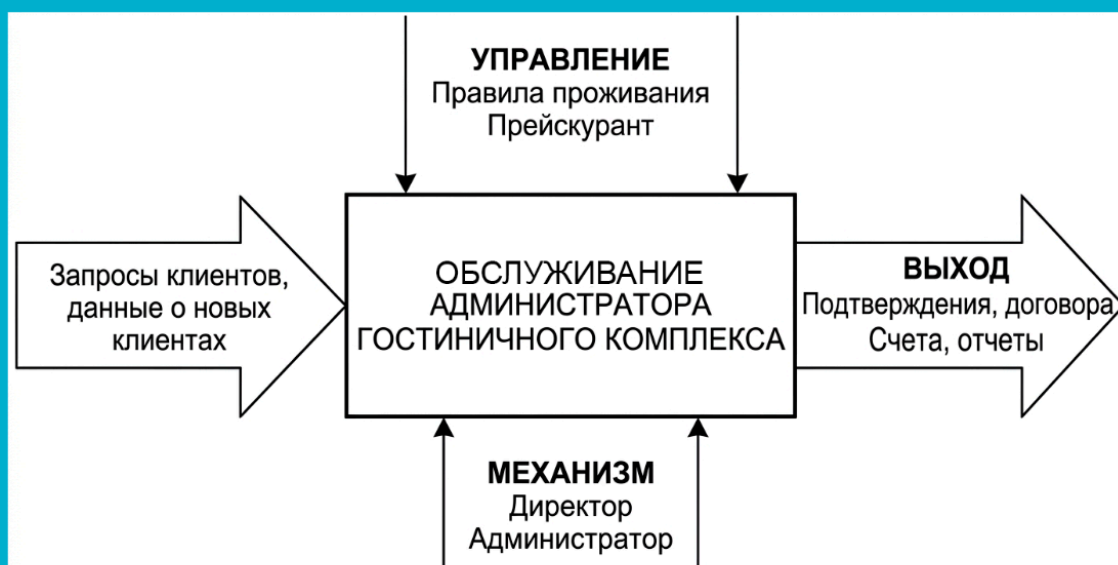
Директор осуществляет общее руководство гостиничным комплексом, определяет стратегию развития, принимает управленческие решения и контролирует финансовую деятельность. Он отвечает за организацию работы всех подразделений и качество предоставляемых услуг.

Администратор играет ключевую роль в работе гостиницы, так как непосредственно взаимодействует с клиентами. В его обязанности входит оформление бронирований, регистрация гостей, заселение и выселение, ведение базы данных, а также консультирование клиентов по услугам гостиницы. Кроме того, администратор координирует работу персонала и решает возникающие вопросы.

Бухгалтерия занимается ведением финансового учета, контролем доходов и расходов, составлением отчетности и расчетом заработной платы сотрудников. Это обеспечивает прозрачность финансовой деятельности гостиницы.

5

Рисунок 22 – Об организационной структуре



Функциональная модель бизнес-процесса

6

Рисунок 23 – Функциональная модель бизнес-процесса

Декомпозиция контекстной диаграммы в нотации IDEF0



7

Рисунок 24 – Декомпозиция контекстной диаграммы в нотации IDEF0

Требования к функционалу

Система должна обеспечивать:

- добавление и редактирование данных о клиентах;
- управление номерным фондом;
- создание и обработку бронирований;
- учет дополнительных услуг;
- расчет стоимости проживания;
- формирование отчетов;
- удобный пользовательский интерфейс.

8

Рисунок 25 – Требования к функционалу

Структура базы данных



9

Рисунок 26 – Структура базы данных

СИСТЕМА УПРАВЛЕНИЯ ОТЕЛЕМ — Панель администратора

Текущее время: 30.04.2026 05:20:01

Номера Бронирования Гости Услуги Отчеты

Фильтры

Тип номера: Все Статус: Все Обновить Добавить номер

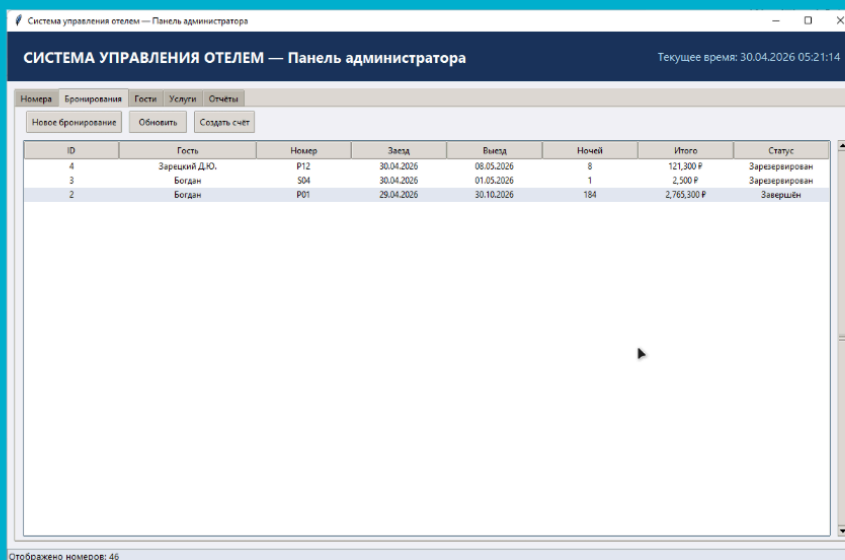
Номер	Тип	Этаж	Вместимость	Цена/ночь	Статус
C01	Комфорт	2	2	4,000 P	Свободен
C02	Комфорт	3	2	4,000 P	Свободен
C03	Комфорт	4	2	4,000 P	Свободен
C04	Комфорт	5	2	4,000 P	Свободен
C05	Комфорт	1	2	4,000 P	Свободен
C06	Комфорт	2	2	4,000 P	Свободен
C07	Комфорт	3	2	4,000 P	Свободен
C08	Комфорт	4	2	4,000 P	Свободен
C09	Комфорт	5	2	4,000 P	Свободен
C10	Комфорт	1	2	4,000 P	Свободен
C11	Комфорт	2	2	4,000 P	Свободен
C12	Комфорт	3	2	4,000 P	Свободен
C13	Комфорт	4	2	4,000 P	Свободен
C14	Комфорт	5	2	4,000 P	Свободен
C15	Комфорт	1	2	4,000 P	Свободен
L01	Люкс	2	4	7,000 P	Свободен
L02	Люкс	3	4	7,000 P	Свободен
L03	Люкс	4	4	7,000 P	Свободен
L04	Люкс	5	4	7,000 P	Свободен
L05	Люкс	1	4	7,000 P	Свободен
L06	Люкс	2	4	7,000 P	Свободен
L07	Люкс	3	4	7,000 P	Свободен
L08	Люкс	4	4	7,000 P	Свободен
P01	Президентский	2	4	15,000 P	Свободен
P02	Президентский	3	4	15,000 P	Свободен

Отображено номеров: 46

Интерфейс вкладки «Номера»

10

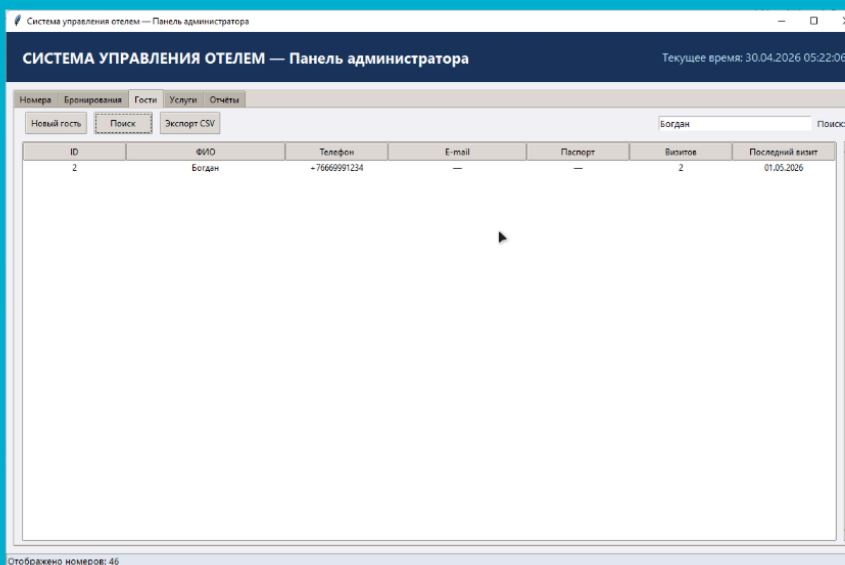
Рисунок 27 – Интерфейс системы «Номера»



Интерфейс вкладки «Бронирование»

11

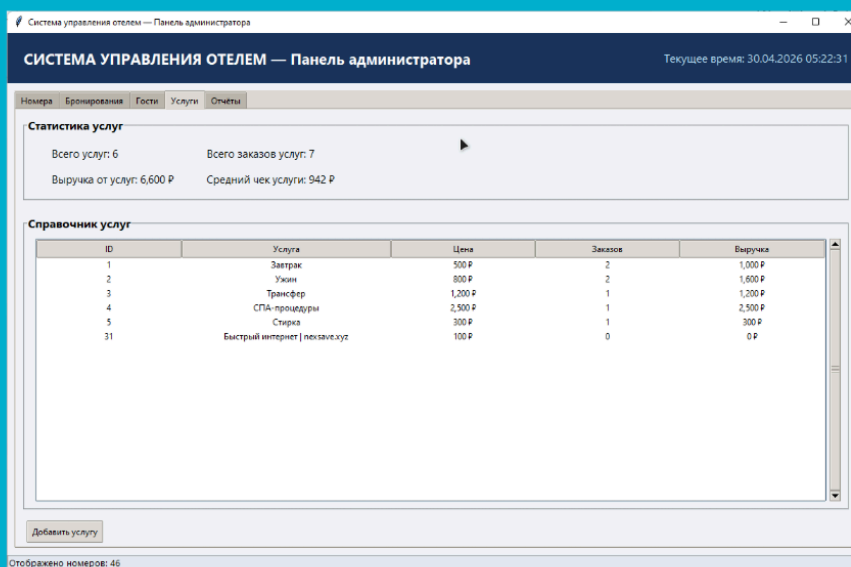
Рисунок 28 – Интерфейс системы «Бронирование»



Интерфейс вкладки «Гости»

12

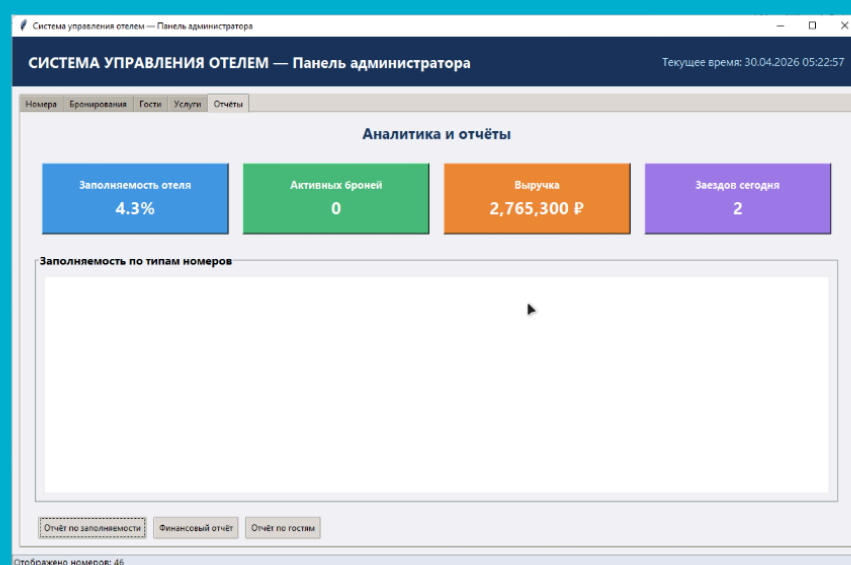
Рисунок 29 – Интерфейс системы «Гости»



Интерфейс вкладки «Услуги»

13

Рисунок 30 – Интерфейс системы «Услуги»



Интерфейс вкладки «Отчёты»

14

Рисунок 31 – Интерфейс системы «Отчеты»

Заключение

Результаты работы:

- Были успешно решены поставленные задачи по разработке автоматизированной информационной системы для автоматизации деятельности администратора гостиничного комплекса
- Была изучена деятельность гостиничного комплекса
- Проведенное функциональное моделирование позволило наглядно представить процессы работы администратора
- Была спроектирована и реализована автоматизированная информационная система

15

Рисунок 32 – Заключение

Министерство науки и высшего образования Российской Федерации Федеральное
государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж им. В.В. Петрова

КУРСОВОЙ ПРОЕКТ
КР 09.02.07.07.000ПЗ

Разработка приложения для автоматизации деятельности администратора гостиничного комплекса

Выполнил: студент гр. 1Веб-42 - Ларионов В.Г.

Руководитель: преподаватель каф. ИСЭ - Фетисова С.Ю.

БАРНАУЛ 2025

Рисунок 33 – Титульный лист